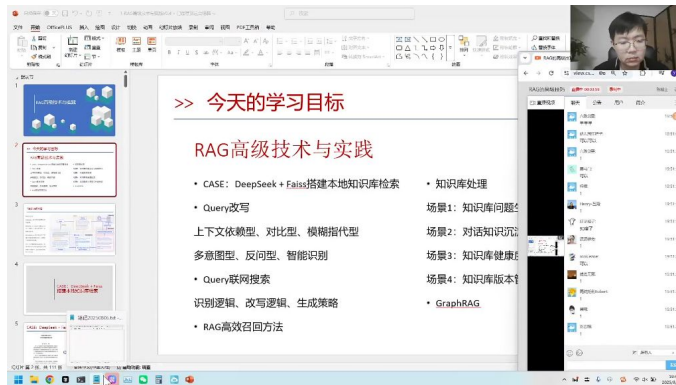


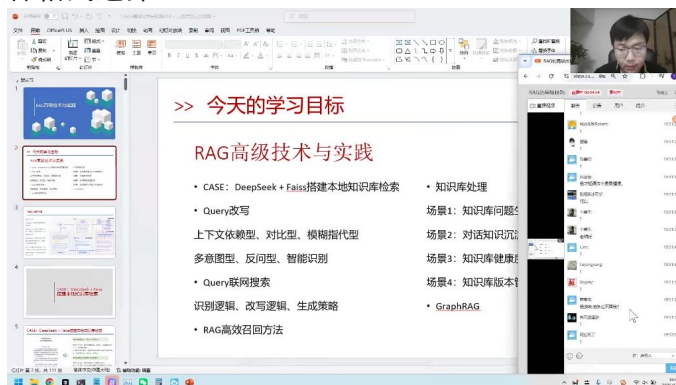
一、RAG高级技巧06:00

1. 设备调试 06:57



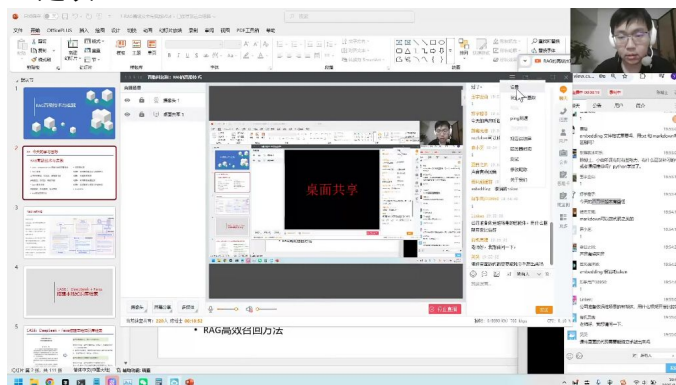
- 课前准备：调试设备并确认学员能正常接收课程内容
- 课件获取：要求学员提前下载课件和代码文件

2. 文件格式选择 07:13



- 格式选择原则：
 - 核心因素：内容解析程度比格式本身更重要
 - PDF/PPT转换：建议转换为markdown格式，相比txt内容损失更小
 - Markdown优势：支持代码块、表格和图像嵌入，布局保留更完整
 - 纯文本局限：txt仅适合无格式要求的纯文本内容

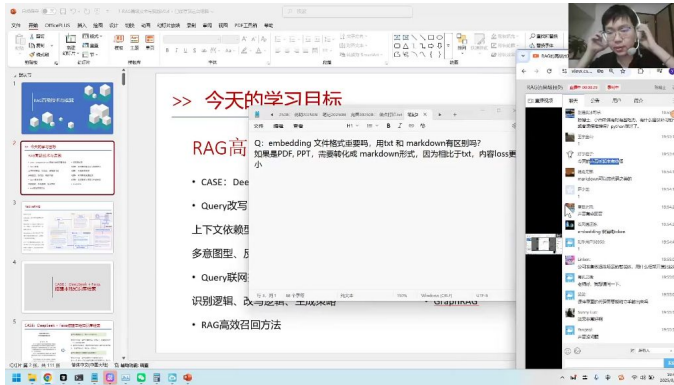
3. 学习建议



- 学习方法论：
 - 聚焦原则：建议先深入掌握当前课程内容而非盲目扩展
 - 问题处理：遇到不理解处可通过AI工具(如DeepSeek)辅助理解
 - 代码实践：不需要手工敲代码，重点在于逻辑理解和结果验证
- AI编程趋势：
 - 行业实践：国外公司已普遍要求使用AI辅助编程

- **关键能力**：逻辑思维能力比语法记忆更重要
- **必要环节**：代码检查和逻辑梳理仍需人工参与

4. 技术问答



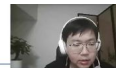
- **开发框架选择**：
 - **常见选项**：LangChain、Graph、千帆Agent等
 - **选择标准**：根据智能体功能需求选择工具链
- **模型部署趋势**：
 - **两极发展**：**旗舰大模型**(如DeepSeek全量版)与**嵌入式小模型**(如1.5B/0.6B)
 - **移动端应用**：本地化小模型在手机端将成为重要场景
- **知识库应用**：
 - **办公集成**：通过API可将规范知识库集成到办公软件
 - **功能扩展**：可实现方案编写指导和提醒功能

5. RAG技术核心

- **必备技能**：RAG是大模型应用开发的核心技术
- **准确率提升**：课程将重点讲解提高回答准确率的方法
- **实践案例**：包含DeepSeek+Faiss本地知识库搭建实例

6. RAG流程回顾 17:41

>> 今天的学习目标



RAG高级技术与实践

- CASE：DeepSeek + Faiss搭建本地知识库检索
- 知识库处理
- Query改写
- 场景1：知识库问题生成与检索优化
- 上下文依赖型、对比型、模糊指代型
- 场景2：对话知识沉淀
- 多意图型、反问型、智能识别
- 场景3：知识库健康度检查
- Query联网搜索
- 场景4：知识库版本管理与性能比较
- 识别逻辑、改写策略、生成策略
- GraphRAG
- RAG高效召回方法

- **核心流程**：分为知识库处理、检索优化和生成策略三个主要环节
- **关键技术**：
 - **Query改写**：用户问题与知识库答案存在句式差异（问句vs陈述句），需要建立改写逻辑
 - **智能识别**：处理上下文依赖型、对比型、模糊指代型、多意图型、反问型等复杂查询
 - **联网搜索**：对于时效性问题（如新闻查询），需识别并触发联网搜索功能

1) 向量数据库 21:25

NativeRAG

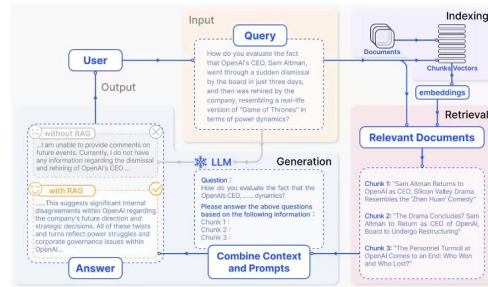
RAG的步骤:

Indexing => 如何更好地把知识存储起来。

Retrieval => 如何在大量的知识中, 找到一小部分有用的, 给到模型参考。

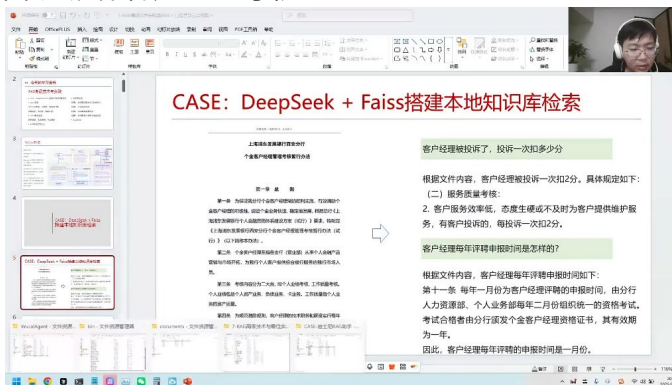
Generation => 如何结合用户的提问和检索到的知识, 让模型生成有用的答案。

这三个步骤虽然看似简单, 但在RAG应用从构建到落地实施的整个过程中, 涉及较多复杂的工作内容。



- 处理阶段:
 - **Indexing**: 将原始文档切分为300-1000字符的chunk (推荐300-500), 转化为**向量存储**
 - **Retrieval**: 计算查询向量与文档向量的cos相似度, 取Top K (如K=3) 最相关片段
 - **Generation**: 将检索结果与用户问题拼接后输入LLM生成最终答案
- 技术细节:
 - **向量维度**: 由embedding模型决定且固定 (如DashScopeEmbeddings)
 - **相似度计算**: 通过cos值排序, 值越大相关性越高
 - **框架实现**: 可使用LangChain封装QA链, 结合FAISS等向量数据库

2) 案例#银行客户经理考核 25:15



- 实现方案:
 - **架构组成**: 使用**LangChain QA链+DeepSeek LLM+FAISS向量库**
 - **关键组件**:
 - **Retriever**: 通过**similarity_search**实现**向量检索**
 - **Chain类型**: 采用Stuff方式拼接检索结果与问题
 - **溯源功能**: 记录文档页码实现答案溯源

● 代码要点:

7. 召回策略 29:38

1) 召回数量设置 30:41

- **参数k的含义**: 代表召回片段的数量, 在向量搜索中通过 `similarity_search(query,k=10)` 设置
 - **典型取值建议**:
 - 常规场景: 4-5个片段
 - 扩展场景: 可增至10个甚至30个
 - 最优实践: 5-10个为最常用范围
 - **注意事项**: 当k值过大 (如30) 时需进行重排序处理, 否则可能影响结果质量
- ## 2) 知识图谱应用 31:03
- **与传统切分的区别**: 知识图谱通过**实体和语义关系连接片段**, 而非独立散落的chunk

- **典型应用场景：**
 - 公务员考试中的申论知识点关联
 - 生物学课程中的概念网络构建
- **实例说明：**以"姚明女儿身高"查询为例，通过"姚明-女儿-身高"的实体关系链实现精准召回
- **技术实现：**使用**Graph RAG**工具建立语义关联网络

8. 重排序模型 33:43

1) BGE-Rerank模型 39:00 ModelScope 中检索

Rerank模型使用

```
import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('BAAI/bge-reranker-large')
model = AutoModelForSequenceClassification.from_pretrained('BAAI/bge-reranker-large')
model.eval()

pairs = [['what is panda?', 'The giant panda is a bear species endemic to China.']]
inputs = tokenizer(pairs, padding=True, truncation=True, return_tensors='pt')
scores = model(**inputs).logits.view(-1).float()
print(scores) # 输出相关性分数 4.9538
```

在BGE-Rerank模型中，相关性分数scores是一个未归一化的对数几率（logits）值，范围没有固定的上限或下限（不像某些模型限制在0-1）。不过BGE-Rerank的分数通常落在以下范围：
 高相关性：3.0~10.0
 中等相关性：0.0~3.0
 低相关性/不相关：负数（如-5.0以下）



- **模型原理：**基于监督学习的序列分类模型，通过人工标注样本学习query-doc相关性打分
- **分数解读：**
 - 高相关：3.0~10.0（如示例中"熊猫是中国熊科动物"得4.9538分）
 - 中等相关：0.0~3.0
 - 不相关：负值（如错误关联"埃菲尔铁塔"得-9分）
- **部署方式：**支持本地化部署，提供base和large两种版本选择
- **计算特点：**相比**召回模型**具有更高复杂度，模型文件通常达2-3GB

2) Cohere Rerank模型 42:59

- **商业API特性：**专为混合检索结果优化设计，输出标准化0-1分数
- **应用示例：**
 - 查询"法国首都在哪"
 - 文档1: "巴黎是法国首都" → 高分
 - 文档2: "柏林是德国首都" → 低分
- **与召回模型对比：**
 - 召回模型：多通路混合策略（关键词/向量），侧重快速初筛
 - 重排模型：精细相关性评估，侧重结果质量优化

9. 查询优化 44:45

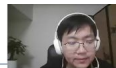
1) Query改写 47:12

- **必要性：**RAG的核心在于"**检索-生成**"，若第一步检索质量差，后续生成质量必然降低。用户问题常为口语化、模糊或含情绪，而知识库内容多为客观陈述，需通过改写实现两者匹配。
- **实现方式：**通过精心设计的Prompt引导LLM完成改写任务，将口语化查询转换为书面化、精确的检索语句。
- **上下文依赖型 49:13**
 - **特征：**用户问题隐含**上下文依赖**（如使用"还"、"其他"等词）
 - **改写方法：**让LLM扮演智能查询助手，分析历史对话后生成包含完整上下文的独立问题
 - **示例：**原问题"还有其他设施吗？" → 改写为"除了疯狂动物城警察局、朱迪景观训练营，疯狂动物城园区还有其他设施吗？"

- 对比型 52:02
 - **特征**：包含比较性词语（如"哪个更好"、"有什么区别"）
 - **改写方法**：识别需对比的多个对象，生成明确的对比查询
 - **示例**：原问题"哪个园区好玩？" → 改写为"比较迪士尼疯狂动物城和蜘蛛侠主题园的游玩体验"
- 模糊指代型 52:56
 - **特征**：含指代不明词汇（如"这个"、"那个"、"他们"）
 - **改写方法**：替换指代名词为具体对象
 - **示例**：原问题"都什么时候开始？" → 改写为"上海迪士尼和香港迪士尼的烟花表演分别什么时候开始？"
- 多意图型 54:53
 - **特征**：单个问题包含多个子问题（常含"和"、"还"等连接词）
 - **改写方法**：分解为多个独立问题，以JSON格式输出
 - **示例**：原问题"门票多少钱？需要提前预约吗？停车费怎么收？" → 拆分为三个独立查询问题
- 反问型 56:24
 - **特征**：含情绪化表达或反问句式
 - **改写方法**：识别真实意图后转为中立陈述
 - **示例**：原问题"这不会也要提前一个月预定吧？" → 改写为"查询上海迪士尼门票是否需要提前一个月预定"

10. 意图识别 59:11

Query改写



Thinking：为什么需要 Query 改写？

RAG 的核心在于“检索-生成”。如果第一步“检索”就走偏了，那么后续的“生成”质量也会降低。

用户提出的问题往往是口语化的、承接上下文的、模糊的，甚至是包含了情绪的。

而知识库里的文本（切片/Chunks）通常是陈述性的、客观的。

=> 需要一个翻译官的角色，将用户的“口语化查询”转换成“书面化、精确的检索语句”

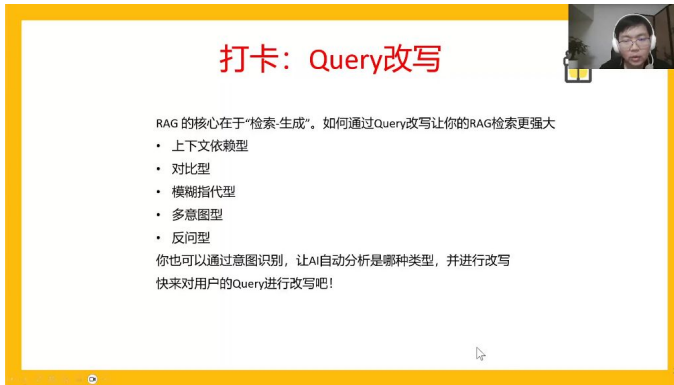
Thinking：如何针对不同类型的Query进行改写？

通过精心设计的 Prompt 来引导 LLM 完成这项任务。

-
- **综合方案**：通过多任务Prompt让LLM同时完成类型识别、查询改写和置信度评估
- **输出格式**：JSON格式包含三个字段：查询类型、改写后查询、置信度（0-1）
- 1) 置信度评估 01:00:45
 - **作用**：反映LLM对改写结果的确定程度（0.8-0.9表示高置信度）
 - **应用场景**：当多种改写类型可能同时匹配时，根据置信度确定优先级
 - **示例输出**：
 - 上下文依赖型：置信度0.95
 - 对比型：置信度0.87
 - 多意图型：置信度0.92

二、Query改写 01:02:22

1. Query改写类型 01:02:27



-
- **核心机制：**RAG采用"检索-生成"架构，Query改写是提升检索效果的关键技术
- **五大改写类型：**
 - **上下文依赖型：**需补充上下文信息（如"还有其他游乐项目吗？"→"除了之前提到的游乐项目之外，还有哪些其他游乐项目？"）
 - **对比型：**需明确比较对象（如"哪个更好？"需补充比较维度）
 - **模糊指代型：**需明确指代对象（如"都适合小朋友吗？"→"哪些产品或活动适合小朋友？"）
 - **多意图型：**需拆分复合问题（如"有什么餐厅？价格怎么样？"→"有哪些餐厅？这些餐厅的价格怎么样？"）
 - **反问型：**需转化为直接疑问（如"这不会也要排队两小时吧？"→"这需要排队两小时吗？"）

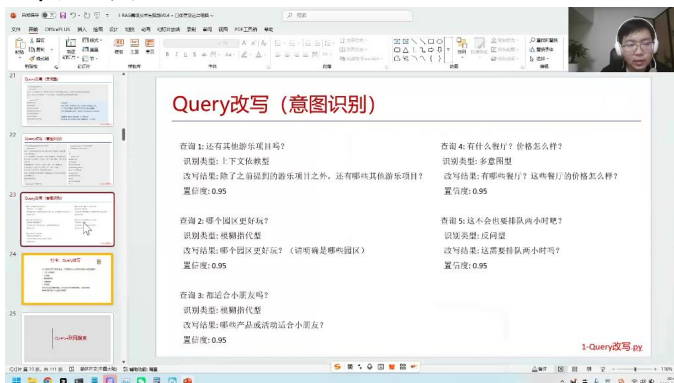
2. 意图识别 01:02:49

- **实现方法：**通过prompt指令让大模型完成类型判断
 - **指令设计：**需包含角色设定、五种类型定义及示例、指定输出格式（类型+改写结果+置信度）
 - **扩展性：**可增加"其他"类型处理未覆盖场景，或让模型自行补充新类型
- **技术选择：**
 - **传统方法：**关键词匹配等规则引擎
 - **大模型方案：**单prompt处理多类型判断，避免多次调用耗时

3. 置信度评估 01:03:04

- **评分机制：**
 - 非数学计算，由大模型基于语义理解给出感性评分
 - 典型高分0.95表示模型对判断结果较确信
- **改进建议：**
 - 可结合向量相似度（如cos值）进行交叉验证
 - 对低置信度结果需设计人工复核流程

4. Query改写实践 01:03:42



-
- **典型场景处理：**

- 模糊查询："哪个园区更好玩？"→明确比较范围（如"上海迪士尼的明日世界和梦幻世界哪个园区更好玩？"）
- 时效查询："今天开放吗？"需保持时间敏感性用于联网检索
- 效果验证：
 - 改写后query应能直接用于向量数据库的top-k检索
 - 检索结果质量与query表述准确性正相关

5. Query改写优化 01:04:22

- 技术选型建议：
 - 优先采用开源方案（如LangChain、LlamaIndex）处理标准流程
 - 定制开发适用于高精度要求的业务场景
- 持续改进：
 - 收集bad case补充到prompt示例库
 - 建立改写效果AB测试机制
 - 对特殊领域可训练专用分类器

三、联网检索场景

类型	关键词特征	示例查询	原因说明
时效性	最新、今天、现在、实时、当前	上海迪士尼乐园今天开放吗？	需获取当前时间的最新信息
价格信息	多少钱、价格、费用、票价	下周六的门票多少钱？	价格信息经常变动，需实时查询
营业信息	营业时间、开放时间、闭园时间、是否开放	迪士尼乐园现在开门吗？	营业状态可能因特殊情况调整
活动信息	活动、表演、演出、节日、庆典	最近有什么特别活动？	活动信息具有时效性和动态性
天气信息	天气、下雨、温度	明天去迪士尼天气怎么样？	天气信息需要实时获取
交通信息	怎么去、交通、地铁、公交	从浦东机场怎么去迪士尼？	交通信息可能因施工、活动等变化
预订信息	预订、预约、购票、订票	需要提前多久预订？	预订政策可能随时调整
实时状态	排队、拥挤、人流量	现在人多不多？	实时状态需即时获取

- 典型场景分类：
 - 时效信息：含"最新"、"今天"等时间关键词（如"上海迪士尼乐园当前排队情况"）
 - 动态数据：价格、营业时间、活动等易变信息（如"下周六门票价格"）
 - 实时状态：人流、天气等瞬时数据（如"现在园区拥挤程度"）
 - 空间信息：交通路线、地理定位等（如"从浦东机场到迪士尼的地铁线路"）

四、联网搜索改写 01:07:46

核心功能1：识别查询是否需要联网搜索

```
def identify_web_search_needs(query, conversation_history):
```

输入：

- query: 用户查询
- conversation_history: 对话历史上下文

输出：

- need_web_search: 是否需要联网搜索
- search_reason: 搜索原因
- confidence: 置信度(0-1)

- 功能定义：通过函数identify_web_search_needs(query, conversation_history)判断查询是否需要联网搜索
- 输入参数：
 - query: 用户当前查询
 - conversation_history: 对话历史上下文

- **输出参数：**
 - need_web_search: 布尔值，表示是否需要联网搜索
 - search_reason: 字符串，说明搜索原因
 - confidence: 0-1之间的置信度评分

1. 时效性信息 01:07:57

- **典型场景：**
 - 时间敏感信息（如"今天开门吗？"、"当前几点几分？"）
 - 营业状态变更（特殊原因导致的调整）
 - 天气、交通等实时信息
- **判断依据：**大模型缺乏实时时间感知能力，需要联网获取最新数据
- **示例：**迪士尼乐园营业查询，需结合当前日期判断是否开放

2. 价格信息 01:08:06

- **特点：**
 - 商品价格变动频繁
 - 需要查询数据库或网络获取实时数据
- **处理方式：**通过置信度评分（如0.7以上）决定是否触发搜索
- **示例：**迪士尼门票价格查询，需获取最新定价和优惠政策

3. 活动信息 01:08:22

- **查询类型：**
 - 特殊活动安排
 - 限时促销信息
 - 演出时间表
- **处理流程：**
 - 判断是否需要联网（置信度>阈值）
 - 对原始查询进行语义改写
 - 生成搜索关键词组合

4. 搜索策略生成 01:08:41

参数名	类型	是否必填	默认值	说明
query	string	是	-	搜索关键词
search_depth	string	否	"basic"	搜索深度，可选值："basic" 或 "advanced"
time_range	string	否	"day"	时间范围，如 "day", "week", "month", "year" 或 "all"
max_results	int	否	5	最大返回结果数量
include_images	bool	否	FALSE	是否包含图片
include_answer	bool	否	FALSE	是否包含由 LLM 生成的简短回答
include_raw_html	bool	否	FALSE	是否包含原始 HTML 内容
topic	string	否	"general"	搜索主题，可选值："general" 或 "news"
domains	list	否	-	限定搜索的域名白名单，如 ["bbc.com", "techcrunch.com"]
exclude_domains	list	否	-	排除的域名黑名单

- **核心要素：**

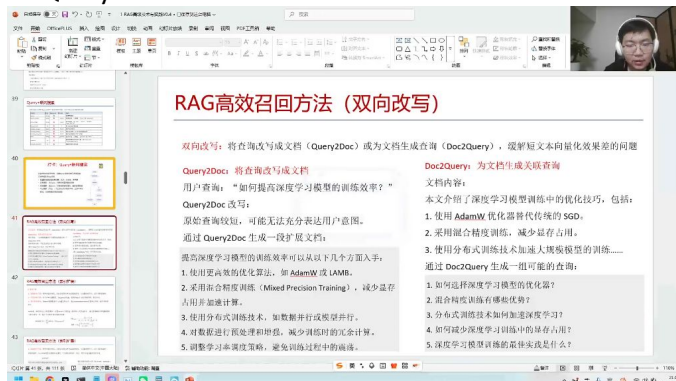
- **主要搜索词**：核心关键词（如"上海迪士尼门票"）
- **扩展搜索词**：同义词和相关词（如"票价"、"优惠"）
- **搜索平台**：推荐使用官方站点或特定搜索引擎
- **时间范围**：根据需求指定（最近一周/一个月等）
- **参数规范**：
 - 使用JSON格式确保结构化输出
 - 包含必填项（query）和选填项（search_depth等）
- **工具集成**：
 - 可调用Tavily等AI搜索引擎
 - 支持通过MCP协议进行标准化调用

五、双向改写01:23:48

1. Query2Doc 01:24:01

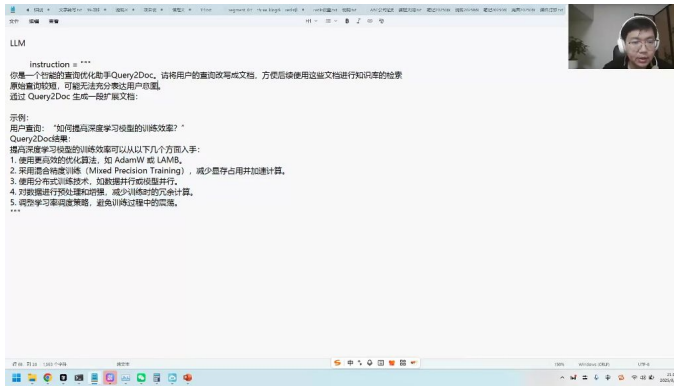
- **核心思想**：将用户简短查询改写成扩展文档，缓解短文本向量化效果差的问题
- **实现方法**：
 - 通过LLM生成包含多个相关维度的扩展文档
 - 示例：用户查询"如何提高深度学习模型的训练效率？"可改写成包含优化算法、混合精度训练、分布式技术等维度的文档
- **应用场景**：当原始查询较短，无法充分表达用户意图时使用
- **注意事项**：
 - 改写结果不是最终答案，而是用于知识库检索的中间产物
 - 计算成本较高，需根据场景价值权衡使用

2. Doc2Query 01:24:24



- **核心思想**：为文档片段生成可能的用户查询，建立文档与问题的多维度联系
- **实现方法**：
 - 针对文档内容预测用户可能提出的各种问题
 - 示例：文档内容涉及深度学习训练优化时，可生成"如何选择优化器？"、"混合精度训练优势？"等问题
- **优势**：
 - 增加文档被检索到的概率
 - 建立问题与文档的深层语义联系
- **典型应用**：知识库预标注，为文档片段添加可能的问题标签

3. 双向改写实践 01:25:07



- 完整流程：
 - 定义LLM指令模板，明确改写要求
 - 提供改写示例（如深度学习训练效率案例）
 - 执行实际查询改写（如"中国有哪些滑雪胜地"）
 - 使用改写结果检索知识库
- 指令设计要点：
 - 明确角色设定（智能查询优化助手）
 - 说明改写目的（便于知识库检索）
 - 提供标准示例格式
- 系统集成：
 - 可作为RAG系统的独立模块
 - 需要与知识库检索、答案生成等模块协同工作
- 成本考量：
 - 改写会增加计算开销
 - 需根据查询价值决定是否启用改写
 - 对时效性、准确性要求高的场景更适合使用

六、索引/扩展01:39:12

1. 离散索引扩展 01:39:22

- 技术实现：使用关键词抽取、实体识别等技术生成离散索引，与向量检索形成互补
- 典型应用：BM25算法作为改进版TF-IDF，通过词频饱和度和文档长度归一化提升相关性排序效果

算法公式：

$$BM25(Q,D) = \sum_{i=1}^n \frac{DF(q_i) \cdot TF_{BM25}(q_i,D)}{(k_1 + 1) \cdot tf}$$
$$TF_{BM25} = \frac{TF}{k_1 \cdot (1 - b + b \cdot \frac{dl}{avgdl}) + tf}$$

- $DF = \log(\frac{N - n + 0.5}{n + 0.5} + 1)$

2. 连续索引扩展 01:39:36

- 模型组合：结合多种向量模型（如OpenAI的Ada、智源的BGE）进行多路召回
- 优势特点：不同模型取长补短，通过embedding计算向量间夹角实现语义匹配

3. 混合索引召回 01:39:48

RAG高效召回方法（索引扩展）



4. 索引扩展

- 1) 离散索引扩展：使用关键词抽取、实体识别等技术生成离散索引，与向量检索互补，提升召回准确性。
- 2) 连续索引扩展：结合多种向量模型（如OpenAI的Ada、智源的BGE）进行多路召回，取长补短。
- 3) 混合索引召回：将BM25等离散索引与向量索引结合，通过Ensemble Retriever实现混合召回，提升召回多样性

BM25是一种经典的文本检索算法，它是TF-IDF（词频-逆文档频率）的改进版本，通过更精细的词频饱和度和文档长度归一化，提升了检索的相关性排序效果。

$$\text{BM25}(Q, D) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \text{TF}_{\text{BM25}}(q_i, D)$$

$$\text{TF}_{\text{BM25}} = \frac{(k_1 + 1) \cdot \text{tf}}{k_1 \cdot (1 - b + b \cdot \frac{\text{dl}}{\text{avgdl}}) + \text{tf}}$$
$$\text{IDF} = \log \left(\frac{N - n + 0.5}{n + 0.5} + 1 \right)$$

-
- 组合方式：将BM25等离散索引与向量索引结合，通过Ensemble Retriever实现
- 应用建议：精确检索场景推荐使用，如文档关键词匹配后再进行向量相似度计算
- 策略选择：不必全部采用，根据实际需求选择1种或组合多种策略

七、Small-to-Big策略01:40:59

1. 小规模内容索引 01:41:07

-
- 内容类型：包括摘要（如论文摘要）、关键句（如技术核心语句）或段落
- 生成技术：通过摘要生成、关键句提取等技术从大规模内容中提取
- 实际案例：Google Notebook LM工具采用此策略，先匹配摘要再关联全文

2. 大规模内容链接 01:41:40

- 关联方式：通过文档ID、URL或指针建立预定义链接
- 内容示例：完整PDF文档（含实验数据）、技术白皮书（含架构细节）
- 优势体现：保持检索效率的同时可获得完整上下文信息

3. 检索机制 01:42:10

- 两阶段检索：先在小规模内容中匹配查询，再链接对应大规模内容
- 上下文补充：将完整文档作为RAG系统输入，提升答案连贯性
- 适用场景：特别适合长文档（如学术论文）和多文档检索场景
- 实践建议：类似Notebook LM的"摘要-全文"分层处理模式最有效

八、知识库处理01:44:53

1. 问题生成 01:44:58

场景1：知识库问题生成与检索优化



Thinking: 当用户提问与知识切片的相似度不高时，能否通过AI为每个知识切片生成可能的问题，通过问题与问题的匹配来提高检索准确度？

TO DO: 对知识库进行问题生成，并进行检索优化

- 自动生成多样化问题：为知识库中的每个知识切片自动生成多种类型、不同难度的问题

`generate_questions_for_chunk()`：为单个知识切片生成基础问题

`generate_diverse_questions()`：生成更多样化的问题（8个）

- 构建双重检索索引：同时构建基于原文内容和生成问题的BM25检索索引
- 检索评估：针对两种检索方式，进行详细的评估

- - **Doc2Query原理**：当用户提问与知识切片相似度不高时，通过AI为每个知识切片生成可能的问题，通过问题与问题的匹配来提高检索准确度
 - **生成方法**：
 - `generate_questions_for_chunk()`：为单个知识切片生成基础问题
 - `generate_diverse_questions()`：生成更多样化的问题（8个）
 - **应用场景**：适用于知识库信息未经充分梳理，用户问题与知识库匹配度不高的情况
- ### 2. 特征工程 01:46:09
- **传统特征工程**：在机器学习阶段对输入模型的特征进行加工处理，以更好发挥模型能力
 - **现代应用**：对现有数据进行预处理后再输入大模型进行推理，与传统特征工程逻辑一致
- ### 3. 数据预处理 01:46:39
- **关键作用**：是知识库处理中的核心环节，直接影响后续模型推理效果
 - **预处理内容**：包括知识切片、问题生成、质量评估等步骤

九、知识库问题生成01:53:21

1. 多样化问题生成 01:53:24

- **问题类型**：包括直接问、间接问、对比问等多种形式
- **示例**：针对"上海迪士尼乐园位于浦东新区，2016年6月16日开园，面积390公顷"的知识切片，可生成：
 - 上海迪士尼乐园何时开园？
 - 中国大陆第一座迪士尼在哪里？
 - 与美国迪士尼相比有什么独特之处？
 - 主要乐园类型有哪些？
 - 占地多少公顷？

2. 问题类型分类 01:53:38

- **输出格式**：采用JSON格式，包含问题内容、问题类型、难易程度等字段
- **质量保证**：通过添加答案验证环节，确保生成问题与知识文档相关性强

3. 问题质量评估 01:53:47

- **评估方法**：让AI预先生成问题对应的答案，验证知识文档能否回答该问题
- **示例**：
 - 问题：上海迪士尼乐园哪年开业？
 - 验证答案：2016年（与文档一致）
 - 问题：迪士尼小镇属于哪个部分？
 - 验证答案：需检查文档是否包含此信息

十、检索优化01:57:26

1. BM25算法 01:57:29

- **检索策略**：采用关键词匹配方法进行问题检索

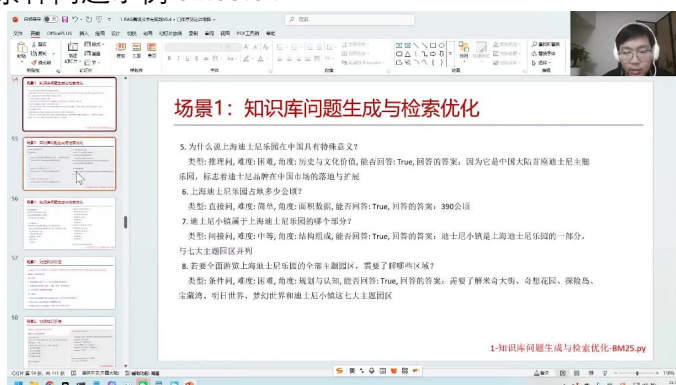
- **双重索引**：同时构建基于原文内容和生成问题的BM25检索索引
2. 检索准确率对比 01:57:56
- **对比实验**：
 - 原文检索：直接匹配用户问题与知识文档
 - 问题检索：匹配用户问题与生成的问题集
 - **结果示例**：
 - "体验最刺激过山车去哪个区域"：问题检索匹配成功，原文检索失败
 - "可以带食物吗"：两种方法均匹配成功
 - "什么时候人气好"：问题检索匹配更准确
3. 检索结果分析 01:59:21
- **优势场景**：问题检索在用户提问与知识文档表述差异大时表现更优
 - **关键因素**：检索效果取决于生成问题的质量和多样性

十一、对话知识抽取 02:01:30

1. 信息提取方法 02:01:33
- **提取内容**：从用户对话中抽取事实性信息、用户需求、常见问题、操作流程、注意事项等
 - **输出格式**：包含知识类型、内容、置信度、来源、关键词和具体分类
2. 知识整合 02:01:53
- **应用场景**：产品上线后从日常对话中持续丰富知识库
 - **工具函数**：从conversation中抽取信息()、批量抽取合并知识点()
3. 知识分类 02:02:22
- **分类维度**：
 - 知识类型（事实性、流程性等）
 - 置信度（信息可靠程度）
 - 来源（原始对话来源）
 - 关键词（便于检索）
 - 具体分类（业务维度分类）

十二、RAG高级技术与实践 02:02:51

1. 知识库问题生成
- 1) 条件问题示例 02:03:04

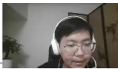


- **推理问示例**："为什么说上海迪士尼乐园在中国具有特殊意义？" 属于历史与文化价值角度，难度为困难
- **直接问示例**："上海迪士尼乐园占地多少公顷？" 属于面积数据角度，难度为简单
- **间接问示例**："迪士尼小镇属于上海迪士尼乐园的哪个部分？" 属于结构组成角度，难度为中等
- **条件问示例**："若要全面游览上海迪士尼乐园的全部主题园区，需要了解哪些区域？" 属于规划与认知角度，难度为困难

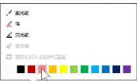
2. 对话知识沉淀 02:03:15

1) 单次对话提取 02:03:19

场景2：对话知识沉淀



示例1: 从单次对话中提取知识
对话内容:
用户: "我想去上海迪士尼乐园玩, 门票多少钱?"
AI: "上海迪士尼乐园的门票价格根据日期有所不同。平日成人票价为399元, 周末和节假日为499元, 儿童票 (1.0-1.4米) 平日为299元, 周末为374元。1.0米以下儿童免费。"
用户: "需要提前预订吗?"
AI: "建议提前预订, 特别是周末和节假日, 可以通过官方网站或第三方平台预订。"
用户: "从浦东机场怎么去?"
AI: "从浦东机场到迪士尼乐园可以乘坐地铁2号线到广兰路站, 然后换乘11号线到迪士尼站, 全程约1小时。也可以打车, 约40分钟。"



2-对话知识沉淀.py

- 提取方法: 从用户与AI的多轮对话中提取知识点, 如门票价格、预订建议、交通方式等
- 提取数量: 示例中从多次对话中提取了22个知识点, 每个知识点都是独立的知识片段

2) 知识分类方法 02:03:35

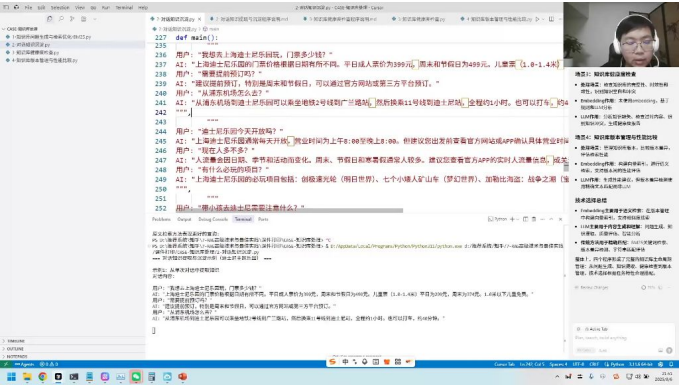
- 事实类型: 如门票价格、开放时间、表演节目、停车场收费标准等具体信息
- 需求类型: 用户表达的需求信息, 如"我想去上海迪士尼", 这类信息需要过滤掉
- 流程类型: 主要涉及交通情况等操作流程信息
- 注意事项类型: 如建议提前预定门票等提醒类信息

3) 知识合并策略 02:03:51

- 过滤原则: 首先过滤掉需求和问题类型的知识, 这些属于用户提问而非答案
- 合并方法: 使用大模型将同类型知识智能合并, 保留重要信息, 去除重复内容
- 合并效果: 可将原本十几个字的短知识点合并为100多字的丰富知识描述
- 事实类合并示例: 将门票价格、开放时间、表演节目等信息合并为完整的事实描述
- 流程类合并: 将不同交通方式信息整合为完整的出行指南

3. 知识库健康检查 02:06:30

1) 冲突知识检测 02:06:36



- Embedding作用: 主要用于语义检索, 在版本管理中构建向量索引支持相似度搜索
- LLM作用: 主要用于内容生成和理解, 包括问题生成、知识提取、质量评估等
- 传统方法应用: BM25关键词检索、版本差异检测、字符串匹配等精确匹配场景
- 完整生命周期: 包含问题生成、知识提取、健康检查和版本管理四个完整环节

4. 知识库版本管理 02:10:45

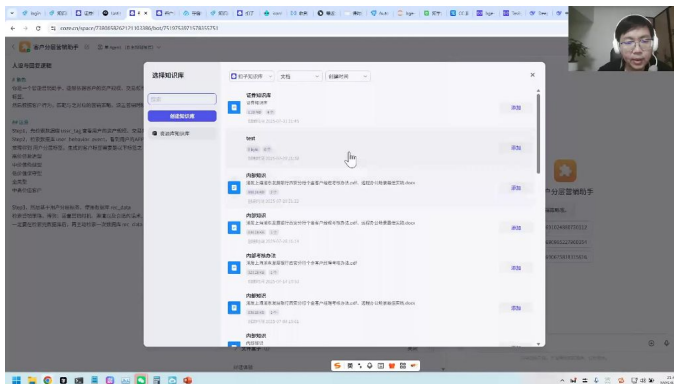
1) 性能评估指标 02:10:58

- 健康检查维度: 通过三大检测维度评估知识库质量, 包括缺少知识检测 (如未覆盖"特别活动"和"停车费"查询)、过期知识检测 (基于时间范围判断, 例如2025年8月6日检测到5月1日-5日的政策已过期)、冲突知识检测 (版本差异导致的内容矛盾)
- 检测方法:

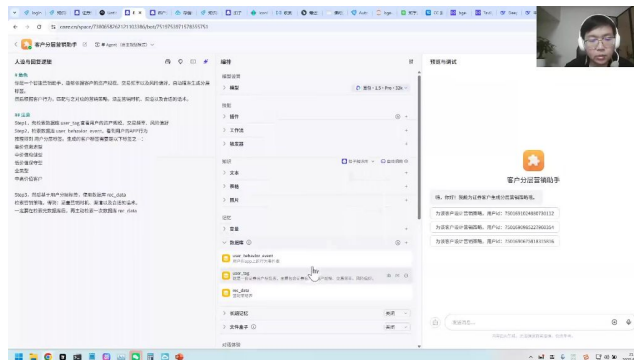
- **关键词匹配法**：通过测试问题与答案关键词的包含关系判断知识完整性（如"上海迪士尼乐园在哪里"需包含"浦东新区"）
- **时间戳比对**：传入当前时间与知识有效期进行新鲜度打分（检测到价格信息和活动信息已过期案例）
- **一致性专家模型**：大模型扮演冲突检测角色，识别版本差异（如两个活动时间描述冲突）
- **版本差异分析**：
 - **量化对比**：版本1含3条知识，版本2扩展至5条，新增2条并优化原有内容
 - **准确率提升**：版本1回答准确率60%（3/5），版本2达100%（5/5）
 - **响应时间**：知识增加可能导致检索时间轻微上升
- **回归测试要求**：需验证旧版本问题在新版本中的通过率（案例显示5个历史问题全部通过）

5. 企业知识库实践 02:12:24

1) 文件夹式管理 02:12:39



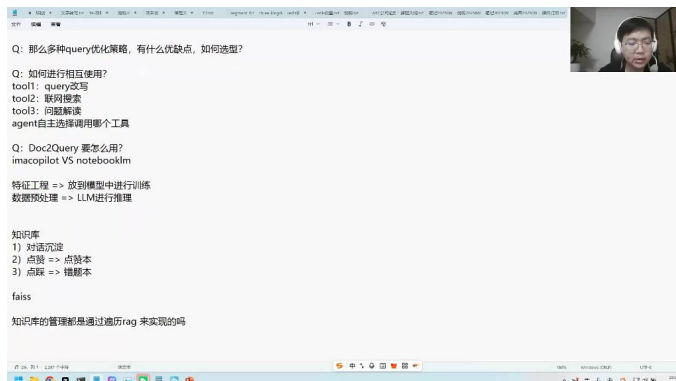
- **多库并存特点**：
 - **分类存储**：企业通常按业务领域划分多个知识库（如证券知识、内部制度、考核办法等）
 - **扁平化结构**：单个知识库采用浅层文件夹结构，类似文件目录管理
 - **部门级应用**：单个部门内部也可能存在多个专用知识库（如营销策略库、客户标签库）



- **典型应用案例**：
 - **客户分层系统**：通过三阶段流程实现精准营销（资产规模分析→行为事件匹配→策略话术生成）
 - **动态检索机制**：需二次检索rec_data确保策略时效性（案例用户ID：7501691024880730112等）
 - **标签体系**：包含"高价值激进型"、"中价值健型"等5类标准标签

6. 技术实现方式 02:19:38

1) 高代码开发 02:19:51



-
- **开发平台选择:** 可使用腾讯智能体开发平台等标准化工具, 也支持企业自主开发个性化解决方案
- **特征工程流程:** 数据预处理⇒特征工程⇒放入模型训练⇒LLM推理
- **知识管理工具:** 包含点赞本(优质内容收集)、错题本(问题修正记录)等功能模块

7. 知识库与RAG区别 02:24:51

1) 知识库与RAG的共性 02:24:54

- **共同基础:** 都是对知识库的应用, 属于知识管理系统的不同环节
- **核心维度:** 包含问题生成、知识沉淀、健康检查、版本性能比较四大管理主题

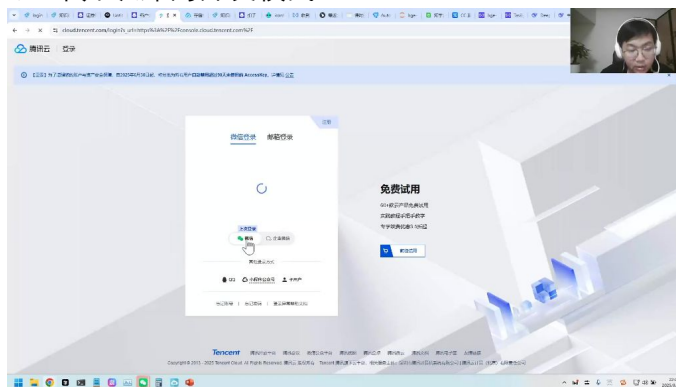
2) 知识库的定义与原始数据 02:25:18

- **本质特征:** 指未经深度处理的原始文档集合, 如企业内部原始资料
- **处理阶段:** 文档需经过"喂入"知识库的预处理过程才能使用

3) RAG作为知识库的使用过程 02:25:29

- **工作流程:**
 - 用户提问查询
 - 知识检索与问题完善 (A阶段)
 - LLM生成最终回答 (B阶段)
- **技术特点:** 实现知识检索 (Retrieval) 与生成 (Generation) 的增强 (Augmented) 组合

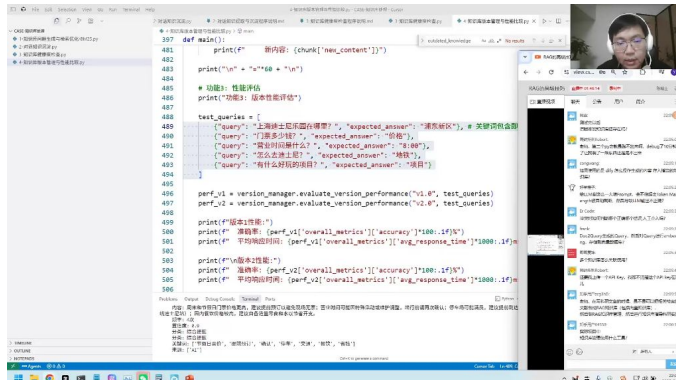
4) 企业中知识库的开发模式 02:25:59



-
- **标准化方案:**
 - 使用大厂产品如腾讯智能体开发平台、COS等
 - 提供文档导入、问答生成等基础功能
 - 通常按年付费, 三年后可买断
- **自主开发:**
 - 解决个性化需求
 - 需实现错题本等高级功能
 - 典型案例包含智能体和知识库两大模块

5) 知识库与向量数据库的关系 02:26:47

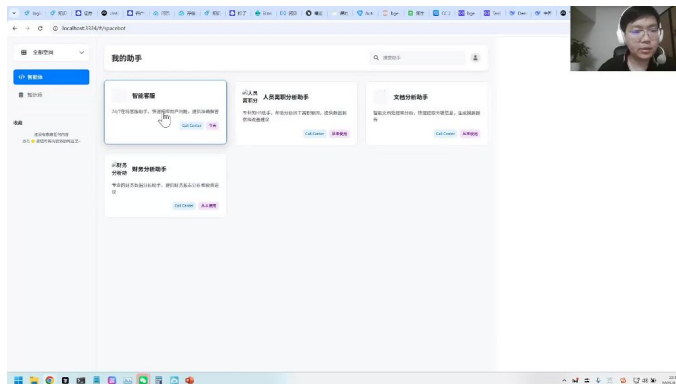
- **功能分工**：知识库存储原始文档，向量数据库负责知识筛选
 - **数学基础**：使用余弦相似度 ($\cos\theta$) 等算法进行向量计算
- 6) 向量数据库的作用与价值 02:26:56
- **核心优势**：能快速处理海量数据的相似度计算
 - **典型应用**：FAISS等工具实现高效语义检索
- 7) 知识库与向量数据库在筛选知识中的应用 02:27:36
- **协作流程**：向量数据库通过数学计算识别最优知识片段，供知识库调用
 - **评估指标**：包含准确率（如浦东新区关键词匹配）、响应时间等性能参数
- 8) 企业自主开发知识库的案例概述 02:27:57



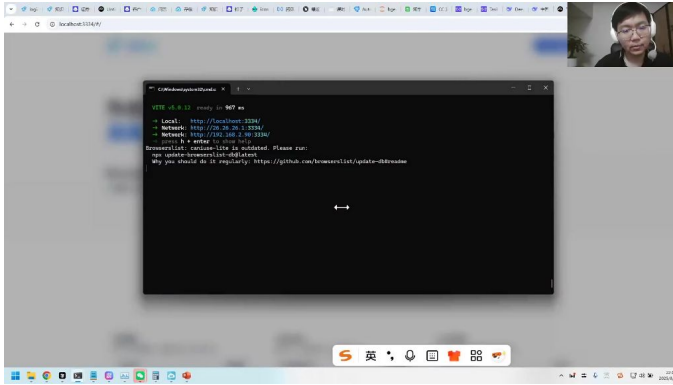
- **开发要点**：
 - 需整合企业特定功能需求
 - 典型评估方法：设置测试问题集（如"上海迪士尼位置"对应"浦东新区"关键词）
 - 准确率计算：采用关键词包含策略或LLM判断
- **版本管理**：支持v1.0/v2.0等多版本性能对比（准确率、响应时间等指标）

8. 本地开发案例 02:28:31

1) 智能体开发 02:29:00



- **开发模式**：采用独立开发平台模式，前端使用Vue框架，后端使用Python进行封装衔接
 - **功能模块**：
 - 智能体管理界面
 - 知识库管理界面
 - 24/7在线客服助手
 - 文档分析助手
 - HR离职分析助手
 - **开发流程**：通过客户访谈梳理需求→生成测试问题→搭建流程→提取共性逻辑
- 2) 知识库管理 02:29:10

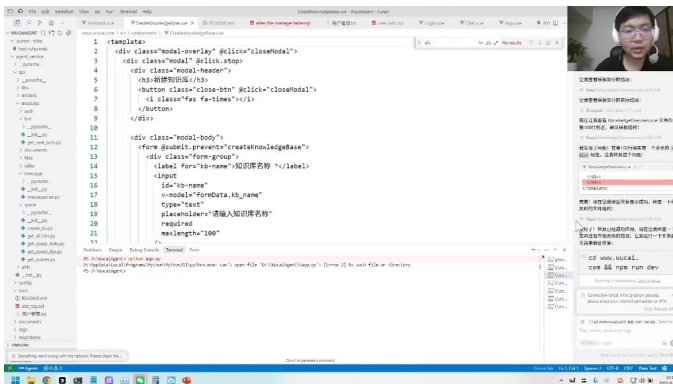


- 管理方式：
 - 支持知识上传和查看
 - 对话内容沉淀功能
 - 点赞生成点赞本
 - 点踩生成错题本
- 技术选型：
 - 使用FAISS进行向量检索
 - 可通过Agent开发平台(如Coze)管理
 - 纯文本占比约10%

3) 高代码与低代码开发模式 02:30:36

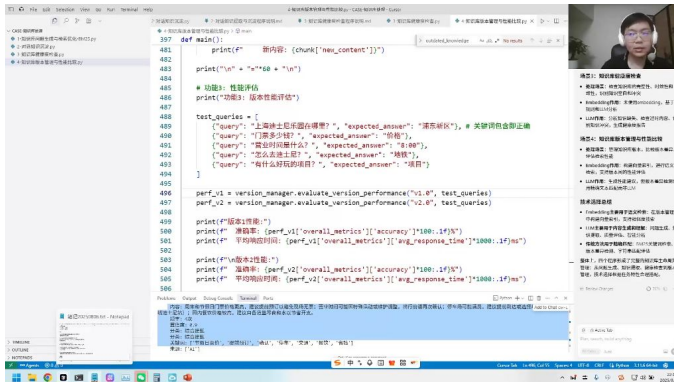
- 低代码特点：
 - 通过可视化拖拽完成开发
 - 提供标准化产品
 - 代表平台：Coze、腾讯智能体平台
- 高代码特点：
 - 需要手动编写大量代码
 - 提供个性化定制产品
 - 使用Cursor等专业开发工具

4) 代码编写体验分享 02:31:12



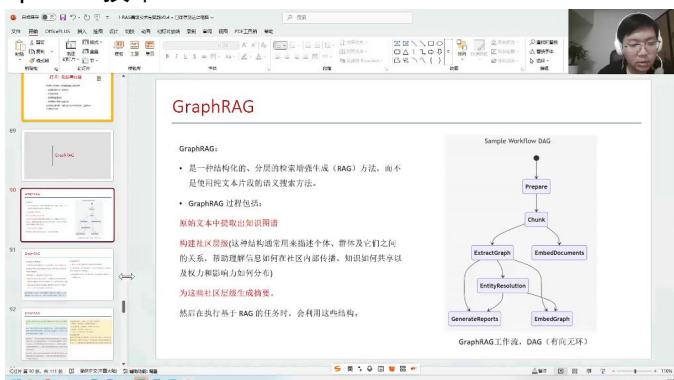
- 开发体验：
 - 单个任务平均耗时约20分钟
 - 需要反复测试和修正
 - 网络延迟会影响开发效率
- 价值体现：
 - 完成产品后成就感显著
 - 可直接交付客户使用
 - 能实现更个性化的功能需求

5) 高代码与低代码的定义 02:32:26



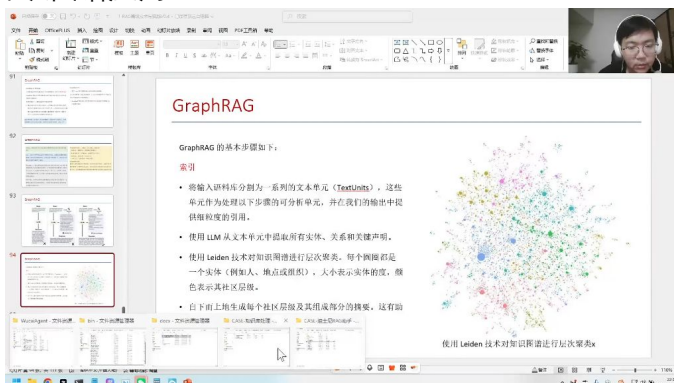
-
- **核心区别：**
 - 低代码：无代码可见，通过配置完成
 - 高代码：需要编写完整代码逻辑
- **适用场景：**
 - 低代码适合标准化需求
 - 高代码适合个性化定制
- **预期管理：**
 - 两种模式实现的最终效果存在差异
 - 需要根据客户需求选择合适的开发方式

9. GraphRAG技术 02:32:51



-
- **结构化方法：**采用结构化的、分层的检索增强生成(RAG)方法，区别于传统纯文本片段的语义搜索方法
- **核心优势：**通过知识图谱处理能提供更高质量的答案，因为对原始文档进行了特殊工程处理

1) 知识图谱应用 02:33:00



-
- **处理流程：**
 - 将输入语料库分割为文本单元(TextUnits)
 - 使用LLM提取实体、关系和关键声明

1) 关羽战胜武将分析 02:42:02

GraphRAG查询



python -m graphrag.query --root ./cases --method local "关羽战胜" python -m graphrag.query --root ./cases --method global "关羽战胜

过哪些武将?"

```
GOOGLE: Gemini Search Response
## 关羽的战绩与地位
关羽，作为三国时期著名的武将，以其忠勇和武艺著称。他在许多战斗中展现了非凡的战斗力，赢得了无数赞誉。以下是对他战绩的一些总结：
### 1. 主要战役
- **白马之战**：关羽在白马之战中，凭借一己之力，斩杀了曹操的猛将颜良，解救了被围困的白马。这场战役不仅展示了关羽的武艺，也体现了他的忠诚。
- **水淹七军**：关羽在樊城之战中，利用汉水的水势，成功淹没了曹操的七军，取得了重大胜利。这场战役是关羽军事才能的集中体现。
- **华容道义释曹操**：关羽在赤壁之战后，在华容道放走了曹操，这一举动虽然违背了刘备的嘱托，但体现了他的义气和宽宏大量。
### 2. 地位与影响
- **忠义的化身**：关羽被后人尊称为“武圣”，他的忠义精神成为后世楷模。他的事迹被广泛传颂，成为中华文化的重要组成部分。
- **军事才能**：关羽在战场上展现出了卓越的军事才能，尤其是在水战方面，他的指挥能力得到了充分的体现。
### 3. 个人品质
- **忠诚**：关羽对刘备的忠诚是众所周知的，即使在面对曹操的重赏和诱惑时，他始终坚守自己的原则。
- **武艺高强**：关羽的武艺高强，他使用的青龙偃月刀更是他的标志性武器。他的武艺在三国时期无人能及。
### 4. 总结
关羽不仅是三国时期的一位杰出武将，更是一位具有高尚品德的楷模。他的事迹和精神，至今仍激励着后人。他的战绩和地位，在历史上留下了浓墨重彩的一笔。
```

胜过哪些武将?"

```
GOOGLE: Gemini Search Response
## 关羽的战绩与地位
关羽，作为三国时期著名的武将，以其忠勇和武艺著称。他在许多战斗中展现了非凡的战斗力，赢得了无数赞誉。以下是对他战绩的一些总结：
### 1. 主要战役
- **白马之战**：关羽在白马之战中，凭借一己之力，斩杀了曹操的猛将颜良，解救了被围困的白马。这场战役不仅展示了关羽的武艺，也体现了他的忠诚。
- **水淹七军**：关羽在樊城之战中，利用汉水的水势，成功淹没了曹操的七军，取得了重大胜利。这场战役是关羽军事才能的集中体现。
- **华容道义释曹操**：关羽在赤壁之战后，在华容道放走了曹操，这一举动虽然违背了刘备的嘱托，但体现了他的义气和宽宏大量。
### 2. 地位与影响
- **忠义的化身**：关羽被后人尊称为“武圣”，他的忠义精神成为后世楷模。他的事迹被广泛传颂，成为中华文化的重要组成部分。
- **军事才能**：关羽在战场上展现出了卓越的军事才能，尤其是在水战方面，他的指挥能力得到了充分的体现。
### 3. 个人品质
- **忠诚**：关羽对刘备的忠诚是众所周知的，即使在面对曹操的重赏和诱惑时，他始终坚守自己的原则。
- **武艺高强**：关羽的武艺高强，他使用的青龙偃月刀更是他的标志性武器。他的武艺在三国时期无人能及。
### 4. 总结
关羽不仅是三国时期的一位杰出武将，更是一位具有高尚品德的楷模。他的事迹和精神，至今仍激励着后人。他的战绩和地位，在历史上留下了浓墨重彩的一笔。
```

- 全局视角：包含吕布、曹仁等5类对手[Data:Reports(299,141)]
- 本地视角：详细战斗过程如"白马之战→颜良"[Data:Entities(1022)]
- 历史影响：奠定"武圣"地位的关键战役

5. 应用场景建议 02:42:25

1) 金融投研案例 02:42:37

- 适用标准：高价值文本分析（招股书/年报）
- 典型场景：上市公司关联网络分析
- 成本考量：需评估ROI，建议文档价值>\$500时采用

十四、RAG高级技术与实践02:42:46

1. GraphRAG介绍

1) GraphRAG基本概念

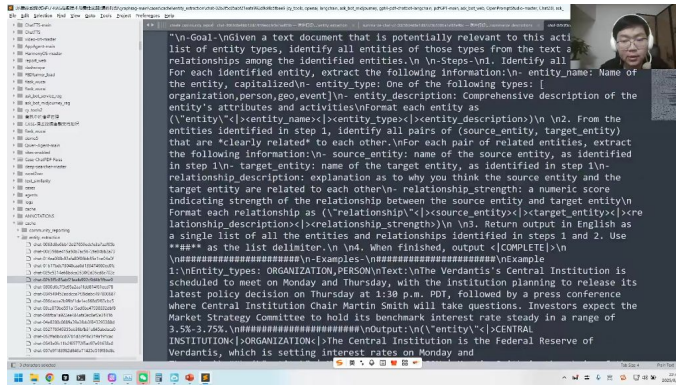
- 核心思想：通过构建知识图谱增强RAG效果，使匹配的实体和关系更丰富
- 关键参数：
 - text_unit_prop: 上下文窗口用于相关文本单位的比例(默认0.5)
 - community_prop: 上下文窗口用于社区报告的比例(默认0.1)
 - conversation_history_max_turns: 对话历史最大轮次(默认5)
 - top_k_mapped_entities: 检索的相关实体数(默认10)
 - top_k_relationships: 引入上下文的网络关系数(默认10)
 - max_tokens: 根据模型标记限制调整(8k模型建议5000，默认12000)

2. GraphRAG应用场景 02:43:15

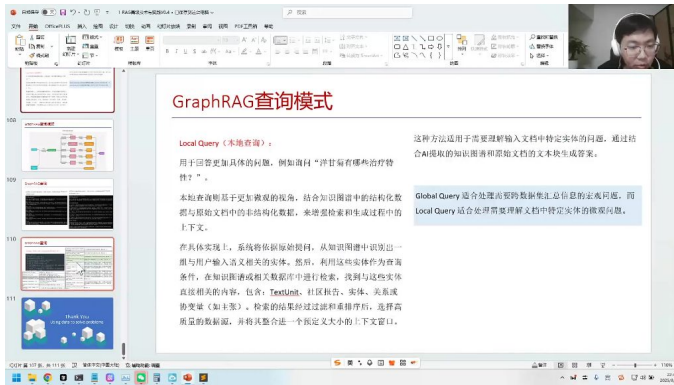
1) 金融领域应用 02:43:31

- 适用性：在金融领域特别有价值，因其需要处理大量实体关系
- 成本考量：其他领域应用需权衡成本效益，预处理一本书约需几十元，每个问题1-2元

2) 知识图谱构建 02:44:13



-
- **组成要素:**
 - 节点(Node): 代表实体(如人物"张飞"、地点"徐州")
 - 边(Edge): 代表实体间关系(如"兄弟关系")
 - 属性: 描述实体特征
- **处理流程:** 对原始文档进行二次处理, 提取实体、关系、社区报告等信息
- 3. GraphRAG数据处理 02:44:55
 - **预处理工作:** 需要大量前期处理, 包括实体定义、关系提取、社区报告整理
 - **数据整理:** 对文章中的实体、关系、社区报告进行系统化整理
- 4. GraphRAG问答质量 02:45:34
 - **优势:** 通过结构化处理使回答更准确, 信息利用率更高
 - **局限性:** 不能完全避免大模型的幻觉问题, 但能显著提升知识匹配度
- 5. 知识库与知识图谱区别 02:46:45
 - **知识库:** 原始文档集合(如三国演义原文)
 - **知识图谱:** 对原始文档的笔记整理, 构建图结构(node,edge)
- 6. GraphRAG替代方案 02:46:55
 - **KAG:** 阿里开发的替代方案
 - **传统RAG:** 如Dify、Coze等平台提供的方案, 成本更低
- 7. 千本Agent方法 02:47:21
 - **特点:** 对数据做切片处理, 使用全量信息回答
 - **对比:** 传统RAG基于top5片段, 信息可能不全
- 8. GraphRAG开源工具 02:47:46
 - **可用性:** 开源代码工具, 基于图理论构建数据模型
 - **检索方式:** 可在key tap上检索相关实现
- 9. 知识存储策略 02:48:46
 - **问题存储:** 可将生成的question与原始文本一起存入知识库
 - **改写技巧:** 通过API对原始信息增加理解性内容
- 10. GraphRAG幻觉问题 02:49:46
 - **改善程度:** 能提升知识利用率, 但不能100%避免幻觉
 - **优势:** 更宏观、完整地利用现有知识
- 11. 查询方法选择 02:50:22



- - **Global Query:** 适合跨数据集汇总信息的宏观问题
 - **Local Query:** 适理解文档特定实体的微观问题
12. 向量查询原理 02:51:26
- 作用: 筛选chunk知识, 通过数学工具(如cos相似度)计算
 - 流程: 问题+筛选出的chunks→LLM生成回答
13. LangChain封装实现 02:53:50
- **MultiQueryRetriever:** LangChain中的检索封装
 - 使用场景: 更适合传统RAG实现, GraphRAG需要更多自定义
14. RAG技术总结 02:54:58
- 1) Query改写技巧 02:55:10
- 必要性: 用户提问与知识库陈述存在表达差异
 - 方法: 意图识别、问题扩展、陈述句转换
- 2) 联网搜索策略 02:55:31
- 应用场景: 问题需要实时外部信息时
 - 实现: 生成搜索策略并整合结果
- 3) 高效召回方法 02:55:45
- **Small to Big:** 从小范围逐步扩大检索
 - 双向检索: query→doc和doc→query结合
 - 混合检索: 结合多种检索策略
15. RAG与知识库关系 02:57:39
- **RAG:** 知识使用过程(检索→增强→生成)
 - 知识库: 原始数据管理
 - 类比: 知识库如藏书, RAG如回答读者问题
16. 向量数据库作用 02:59:10
- 核心功能: 相似度计算引擎
 - 替代方案: 可用纯关键词检索(非语义检索)

十五、知识小结

知识点	核心内容	实践技巧/易混淆点	重要性
RAG基础概念	检索增强生成技术原理: 数据存贮(index)→文档切分(chunk)→向量化→相似度计算→答案生成	chunk size通常300-1000之间(默认800), 300-500颗粒度更细	★★★★★
查询改写策略	5种改写类型: 上下文依赖型/对比型/模糊指代型/多意图型/反问型	需通过prompt工程实现意图识别, JSON格式输出改写结果+置信度	★★★★

混合检索方法	结合关键词检索+向量检索的多通路召回策略	先用简单策略快速召回（如关键词），再用复杂模型重排（如BGE Reranker）	★★★★
知识库优化	双向优化：query→doc生成扩展问题 / doc→query预生成可能提问	预处理阶段生成问答对可提升后续检索准确率20%+	★★★★
知识图谱应用	Graph RAG通过实体识别+关系抽取构建语义网络	相比传统RAG：回答质量↑但成本高（预处理需30分钟/\$100）	★★★
联网搜索优化	8类需联网场景：时间/价格/营业状态/活动/天气/交通/预定/新闻	需生成搜索关键词+平台建设+时间范围三要素	★★★★
知识库健康检查	三维度检测：缺失知识/过期知识/冲突知识	通过测试问题集验证，版本对比准确率变化（如60%→100%）	★★★
Small-to-Big策略	先匹配摘要(small)再关联全文(big)	类似Google Notebook LM的实现方式，降低长文档处理成本	★★★
运维智能体开发	框架选择：LangGraph/千帆Agent等	关键在工具箱设计而非框架本身	★★★
AI编程规范	逻辑设计 > 语法记忆，需关注：代码检查+需求拆解	禁止手敲代码，强调用AI生成（如DeepSeek）	★★★★

以下为AI生成的图文笔记的内容

一、RAG高级技巧06:00

1. 设备调试 06:57

>> 今天的学习目标

RAG高级技术与实践

- CASE: DeepSeek + Faiss搭建本地知识库检索
- Query改写
- Query联网搜索
- RAG高效召回方法
- 知识库处理
- GraphRAG

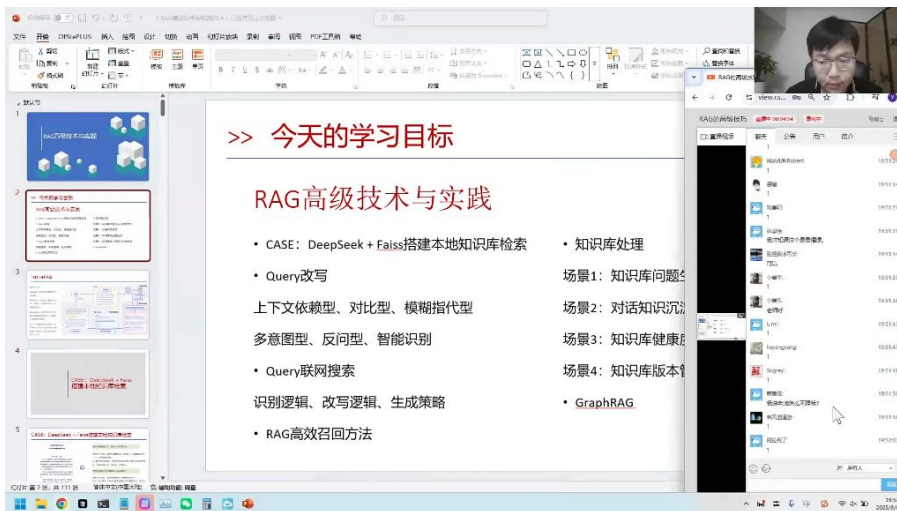
上下文依赖型、对比型、模糊指代型
多意图型、反问型、智能识别

识别逻辑、改写逻辑、生成策略

场景1: 知识库问题
场景2: 对话知识检索
场景3: 知识库健康检查
场景4: 知识库版本管理

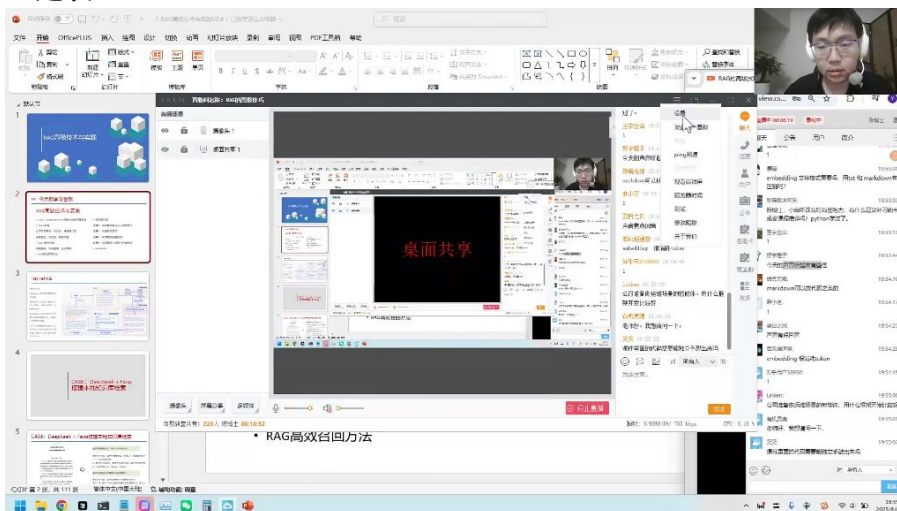
- 课前准备：调试设备并确认学员能正常接收课程内容
- 课件获取：要求学员提前下载课件和代码文件

2. 文件格式选择 07:13



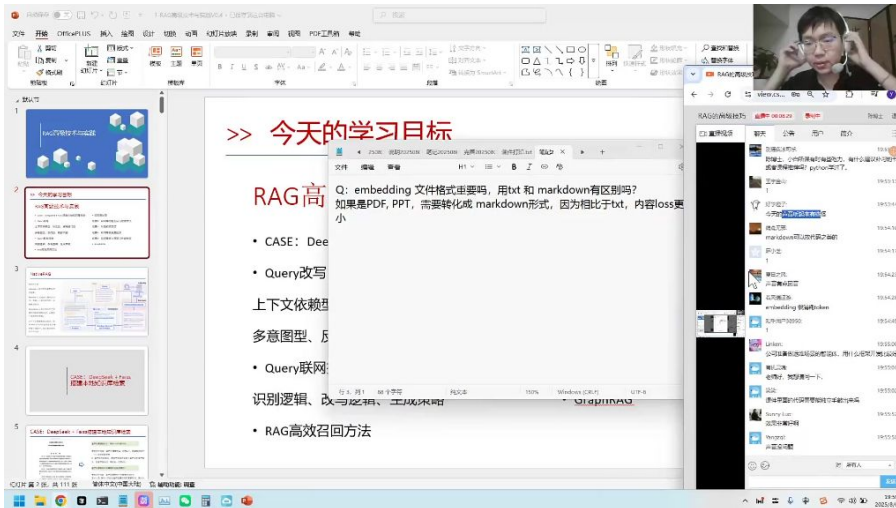
- 格式选择原则：
 - 核心因素：内容解析程度比格式本身更重要
 - PDF/PPT转换：建议转换为markdown格式，相比txt内容损失更小
 - Markdown优势：支持代码块、表格和图像嵌入，布局保留更完整
 - 纯文本局限：txt仅适合无格式要求的纯文本内容

3. 学习建议



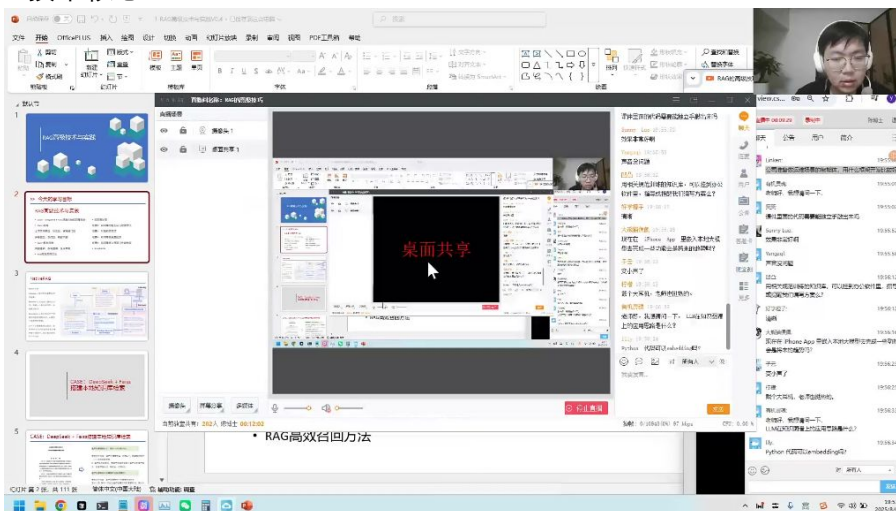
- 学习方法论：
 - 聚焦原则：建议先深入掌握当前课程内容而非盲目扩展
 - 问题处理：遇到不理解处可通过AI工具(如DeepSeek)辅助理解
 - 代码实践：不需要手工敲代码，重点在于逻辑理解和结果验证
- AI编程趋势：
 - 行业实践：国外公司已普遍要求使用AI辅助编程
 - 关键能力：逻辑思维能力比语法记忆更重要
 - 必要环节：代码检查和逻辑梳理仍需人工参与

4. 技术问答



- 开发框架选择:
 - 常见选项: LangChain、Graph、千帆Agent等
 - 选择标准: 根据智能体功能需求选择工具链
- 模型部署趋势:
 - 两极发展: 旗舰大模型(如DeepSeek全量版)与嵌入式小模型(如1.5B/0.6B)
 - 移动端应用: 本地化小模型在手机端将成为重要场景
- 知识库应用:
 - 办公集成: 通过API可将规范知识库集成到办公软件
 - 功能扩展: 可实现方案编写指导和提醒功能

5. RAG技术核心



- 必备技能: RAG是大模型应用开发的核心技术
- 准确率提升: 课程将重点讲解提高回答准确率的方法
- 实践案例: 包含DeepSeek+Faiss本地知识库搭建实例

6. RAG流程回顾 17:41

>> 今天的学习目标

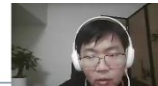


RAG高级技术与实践

- CASE: DeepSeek + Faiss搭建本地知识库检索
- Query改写
- 上下文依赖型、对比型、模糊指代型
- 多意图型、反问型、智能识别
- Query联网搜索
- 识别逻辑、改写逻辑、生成策略
- RAG高效召回方法
- 知识库处理
- 场景1: 知识库问题生成与检索优化
- 场景2: 对话知识沉淀
- 场景3: 知识库健康度检查
- 场景4: 知识库版本管理与性能比较
- GraphRAG

- - 核心流程: 分为知识库处理、检索优化和生成策略三个主要环节
 - 关键技术:
 - **Query改写**: 用户问题与知识库答案存在句式差异 (问句vs陈述句), 需要建立改写逻辑
 - **智能识别**: 处理上下文依赖型、对比型、模糊指代型、多意图型、反问型等复杂查询
 - **联网搜索**: 对于时效性问题 (如新闻查询), 需识别并触发联网搜索功能
- 1) 向量数据库 21:25

NativeRAG



RAG的步骤:

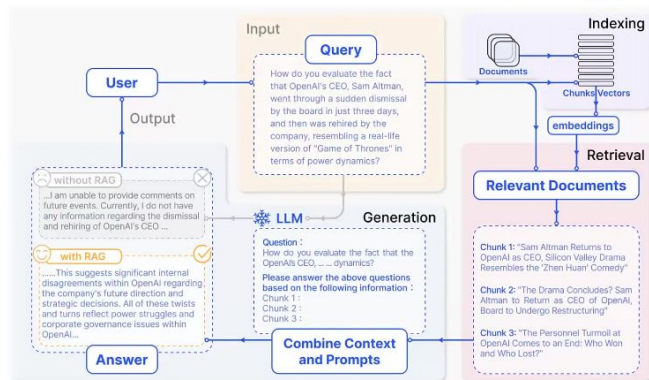
Indexing => 如何更好地把知识存起来。

Retrieval => 如何在大量的知识中, 找到一小部分有用的, 给到模型参考。

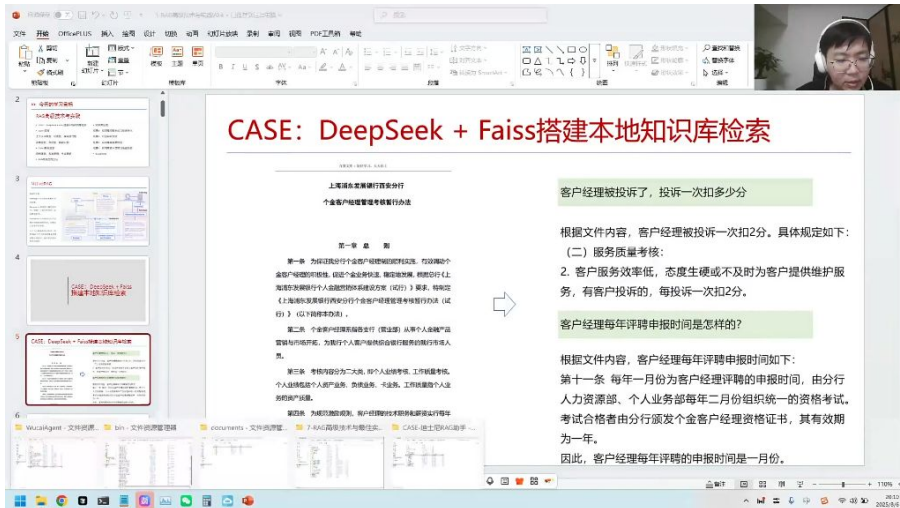
Generation => 如何结合用户的提问和检索到的知识, 让模型生成有用的答案。

这三个步骤虽然看似简单, 但在 RAG 应用从构建到落地实施的整个过程中, 涉及较多复杂

的工作内容。

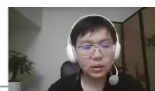


- - 处理阶段:
 - **Indexing**: 将原始文档切分为300-1000字符的chunk (推荐300-500), 转化为向量存储
 - **Retrieval**: 计算查询向量与文档向量的cos相似度, 取Top K (如K=3) 最相关片段
 - **Generation**: 将检索结果与用户问题拼接后输入LLM生成最终答案
 - 技术细节:
 - **向量维度**: 由embedding模型决定且固定 (如DashScopeEmbeddings)
 - **相似度计算**: 通过cos值排序, 值越大相关性越高
 - **框架实现**: 可使用LangChain封装QA链, 结合FAISS等向量数据库
- 2) 案例#银行客户经理考核 25:15



- 实现方案：
 - 架构组成：使用LangChain QA链+DeepSeek LLM+FAISS向量库
 - 关键组件：
 - Retriever：通过similarity_search实现向量检索
 - Chain类型：采用Stuff方式拼接检索结果与问题
 - 溯源功能：记录文档页码实现答案溯源
- 代码要点：
- 7. 召回策略 29:38
 - 1) 召回数量设置 30:41
 - 参数k的含义：代表召回片段的数量，在向量搜索中通过similarity_search(query,k = 10)设置
 - 典型取值建议：
 - 常规场景：4-5个片段
 - 扩展场景：可增至10个甚至30个
 - 最优实践：5-10个为最常用范围
 - 注意事项：当k值过大（如30）时需进行重排序处理，否则可能影响结果质量
 - 2) 知识图谱应用 31:03
 - 与传统切分的区别：知识图谱通过实体和语义关系连接片段，而非独立散落的chunk
 - 典型应用场景：
 - 公务员考试中的申论知识点关联
 - 生物学课程中的概念网络构建
 - 实例说明：以"姚明女儿身高"查询为例，通过"姚明-女儿-身高"的实体关系链实现精准召回
 - 技术实现：使用Graph RAG工具建立语义关联网络
- 8. 重排序模型 33:43
 - 1) BGE-Rerank模型 39:00

Rerank模型使用



```
import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('BAAI/bge-reranker-large')
model = AutoModelForSequenceClassification.from_pretrained('BAAI/bge-reranker-large')
model.eval()

pairs = [['what is panda?', 'The giant panda is a bear species endemic to China.']]
inputs = tokenizer(pairs, padding=True, truncation=True, return_tensors='pt')
scores = model(**inputs).logits.view(-1).float()
print(scores) # 输出相关性分数 4.9538
```

在BGE-Rerank模型中，相关性分数scores是一个未归一化的对数几率（logits）值，范围没有固定的上限或下限（不像某些模型限制在0-1）。不过BGE-Rerank的分数通常落在以下范围：

高相关性：3.0~10.0

中等相关性：0.0~3.0

低相关性/不相关：负数（如-5.0以下）

[bge-reranker.py](#)

- 模型原理：基于监督学习的序列分类模型，通过人工标注样本学习query-doc相关性打分
- 分数解读：
 - 高相关：3.0~10.0（如示例中"熊猫是中国熊科动物"得4.9538分）
 - 中等相关：0.0~3.0
 - 不相关：负值（如错误关联"埃菲尔铁塔"得-9分）
- 部署方式：支持本地化部署，提供base和large两种版本选择
- 计算特点：相比召回模型具有更高复杂度，模型文件通常达2-3GB

2) Cohere Rerank模型 42:59

- 商业API特性：专为混合检索结果优化设计，输出标准化0-1分数
- 应用示例：
 - 查询"法国首都都在哪"
 - 文档1: "巴黎是法国首都" → 高分
 - 文档2: "柏林是德国首都" → 低分
- 与召回模型对比：
 - 召回模型：多通路混合策略（关键词/向量），侧重快速初筛
 - 重排模型：精细相关性评估，侧重结果质量优化

9. 查询优化 44:45

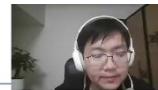
1) Query改写 47:12

- 必要性：RAG的核心在于"检索-生成"，若第一步检索质量差，后续生成质量必然降低。用户问题常为口语化、模糊或含情绪，而知识库内容多为客观陈述，需通过改写实现两者匹配。
- 实现方式：通过精心设计的Prompt引导LLM完成改写任务，将口语化查询转换为书面化、精确的检索语句。
- 上下文依赖型 49:13
 - 特征：用户问题隐含上下文依赖（如使用"还"、"其他"等词）
 - 改写方法：让LLM扮演智能查询助手，分析历史对话后生成包含完整上下文的独立问题
 - 示例：原问题"还有其他设施吗？" → 改写为"除了疯狂动物城警察局、朱迪景观训练营，疯狂动物城园区还有其他设施吗？"
- 对比型 52:02
 - 特征：包含比较性词语（如"哪个更好"、"有什么区别"）
 - 改写方法：识别需对比的多个对象，生成明确的对比查询
 - 示例：原问题"哪个园区好玩？" → 改写为"比较迪士尼疯狂动物城和蜘蛛侠主题园的游玩体验"

- 模糊指代型 52:56
 - **特征**: 含指代不明词汇 (如"这个"、"那个"、"他们")
 - **改写方法**: 替换指代名词为具体对象
 - **示例**: 原问题"都什么时候开始?" → 改写为"上海迪士尼和香港迪士尼的烟花表演分别什么时候开始?"
- 多意图型 54:53
 - **特征**: 单个问题包含多个子问题 (常含"和"、"还"等连接词)
 - **改写方法**: 分解为多个独立问题, 以JSON格式输出
 - **示例**: 原问题"门票多少钱? 需要提前预约吗? 停车费怎么收?" → 拆分为三个独立查询问题
- 反问型 56:24
 - **特征**: 含情绪化表达或反问句式
 - **改写方法**: 识别真实意图后转为中立陈述
 - **示例**: 原问题"这不会也要提前一个月预定吧?" → 改写为"查询上海迪士尼门票是否需要提前一个月预定"

10. 意图识别 59:11

Query改写



Thinking: 为什么需要 Query 改写?

RAG 的核心在于“检索-生成”。如果第一步“检索”就走偏了, 那么后续的“生成”质量也会降低。

用户提出的问题往往是口语化的、承接上下文的、模糊的, 甚至是包含了情绪的。

而知识库里的文本 (切片/Chunks) 通常是陈述性的、客观的。

=> 需要一个翻译官的角色, 将用户的“口语化查询”转换成“书面化、精确的检索语句”

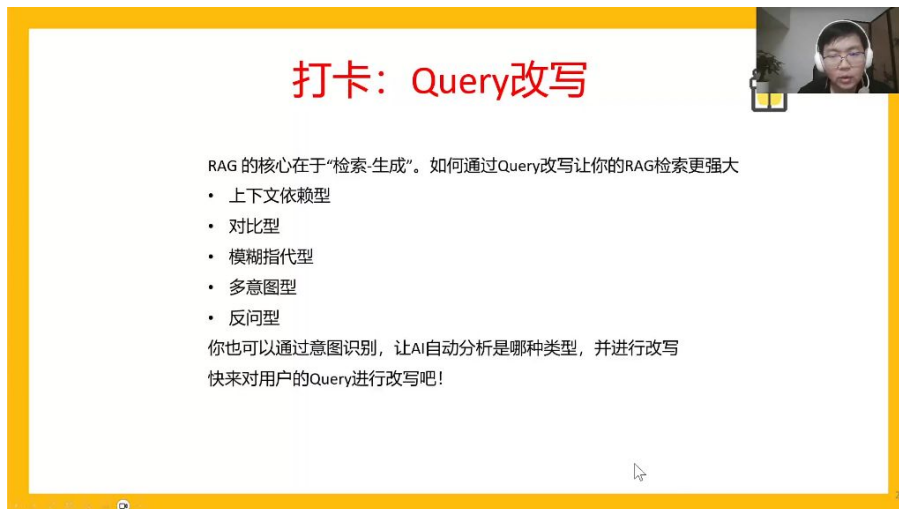
Thinking: 如何针对不同类型的Query进行改写?

通过精心设计的 Prompt 来引导 LLM 完成这项任务。

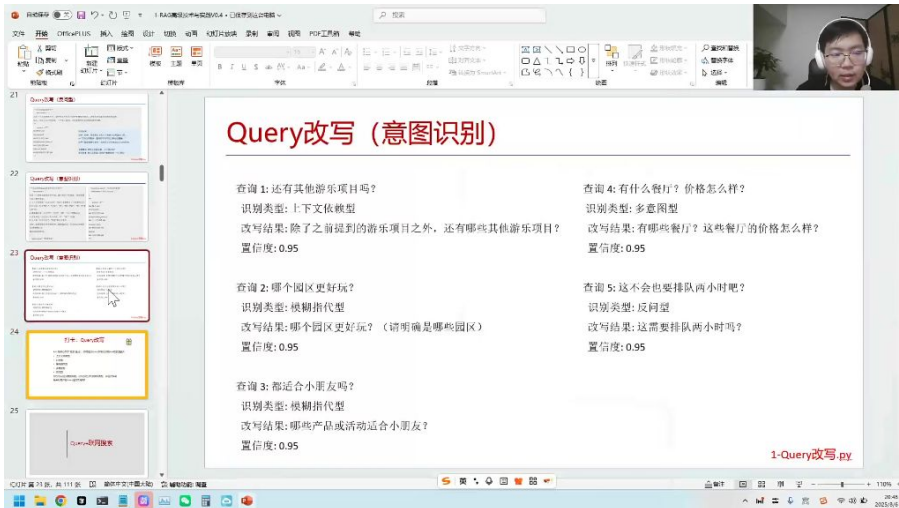
-
- **综合方案**: 通过多任务Prompt让LLM同时完成类型识别、查询改写和置信度评估
- **输出格式**: JSON格式包含三个字段: 查询类型、改写后查询、置信度 (0-1)
- 1) 置信度评估 01:00:45
 - **作用**: 反映LLM对改写结果的确定程度 (0.8-0.9表示高置信度)
 - **应用场景**: 当多种改写类型可能同时匹配时, 根据置信度确定优先级
 - **示例输出**:
 - 上下文依赖型: 置信度0.95
 - 对比型: 置信度0.87
 - 多意图型: 置信度0.92

二、Query改写 01:02:22

1. Query改写类型 01:02:27



-
- **核心机制：** RAG采用"检索-生成"架构，Query改写是提升检索效果的关键技术
- **五大改写类型：**
 - **上下文依赖型：** 需补充上下文信息（如"还有其他游乐项目吗？"→"除了之前提到的游乐项目之外，还有哪些其他游乐项目？"）
 - **对比型：** 需明确比较对象（如"哪个更好？"需补充比较维度）
 - **模糊指代型：** 需明确指代对象（如"都适合小朋友吗？"→"哪些产品或活动适合小朋友？"）
 - **多意图型：** 需拆分复合问题（如"有什么餐厅？价格怎么样？"→"有哪些餐厅？这些餐厅的价格怎么样？"）
 - **反问型：** 需转化为直接疑问（如"这不会也要排队两小时吧？"→"这需要排队两小时吗？"）
- 2. 意图识别 01:02:49
 - **实现方法：** 通过prompt指令让大模型完成类型判断
 - **指令设计：** 需包含角色设定、五种类型定义及示例、指定输出格式（类型+改写结果+置信度）
 - **扩展性：** 可增加"其他"类型处理未覆盖场景，或让模型自行补充新类型
 - **技术选择：**
 - **传统方法：** 关键词匹配等规则引擎
 - **大模型方案：** 单prompt处理多类型判断，避免多次调用耗时
- 3. 置信度评估 01:03:04
 - **评分机制：**
 - 非数学计算，由大模型基于语义理解给出感性评分
 - 典型高分0.95表示模型对判断结果较确信
 - **改进建议：**
 - 可结合向量相似度（如cos值）进行交叉验证
 - 对低置信度结果需设计人工复核流程
- 4. Query改写实践 01:03:42

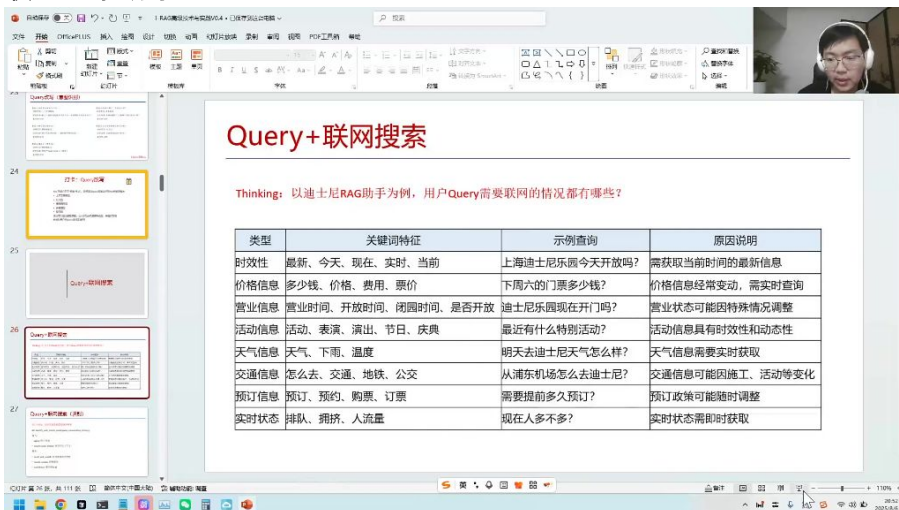


- 典型场景处理:
 - 模糊查询: "哪个园区更好玩?" → 明确比较范围 (如"上海迪士尼的明日世界和梦幻世界哪个园区更好玩?")
 - 时效查询: "今天开放吗?" 需保持时间敏感性用于联网检索
- 效果验证:
 - 改写后query应能直接用于向量数据库的top-k检索
 - 检索结果质量与query表述准确性正相关

5. Query改写优化 01:04:22

- 技术选型建议:
 - 优先采用开源方案 (如LangChain、LlamaIndex) 处理标准流程
 - 定制开发适用于高精度要求的业务场景
- 持续改进:
 - 收集bad case补充到prompt示例库
 - 建立改写效果AB测试机制
 - 对特殊领域可训练专用分类器

三、联网检索场景



- 典型场景分类:
 - 时效信息: 含"最新"、"今天"等时间关键词 (如"上海迪士尼乐园当前排队情况")
 - 动态数据: 价格、营业时间、活动等易变信息 (如"下周六门票价格")
 - 实时状态: 人流、天气等瞬时数据 (如"现在园区拥挤程度")

- **空间信息**：交通路线、地理定位等（如"从浦东机场到迪士尼的地铁线路"）

四、联网搜索改写 01:07:46

Query+联网搜索（识别）



核心功能1：识别查询是否需要联网搜索

```
def identify_web_search_needs(query, conversation_history):
```

输入：

- query: 用户查询
- conversation_history: 对话历史上下文

输出：

- need_web_search: 是否需要联网搜索
- search_reason: 搜索原因
- confidence: 置信度(0-1)

-
- **功能定义**：通过函数identify_web_search_needs(query, conversation_history)判断查询是否需要联网搜索
- **输入参数**：
 - query: 用户当前查询
 - conversation_history: 对话历史上下文
- **输出参数**：
 - need_web_search: 布尔值，表示是否需要联网搜索
 - search_reason: 字符串，说明搜索原因
 - confidence: 0-1之间的置信度评分

1. 时效性信息 01:07:57

- **典型场景**：
 - 时间敏感信息（如"今天开门吗？"、"当前几点几分？"）
 - 营业状态变更（特殊原因导致的调整）
 - 天气、交通等实时信息
- **判断依据**：大模型缺乏实时时间感知能力，需要联网获取最新数据
- **示例**：迪士尼乐园营业查询，需结合当前日期判断是否开放

2. 价格信息 01:08:06

- **特点**：
 - 商品价格变动频繁
 - 需要查询数据库或网络获取实时数据
- **处理方式**：通过置信度评分（如0.7以上）决定是否触发搜索
- **示例**：迪士尼门票价格查询，需获取最新定价和优惠政策

3. 活动信息 01:08:22

打卡：Query+联网搜索

以迪士尼RAG助手为例，搭建Query+联网搜索的改写功能，方便后续进行RAG问答

- 设置联网搜索的使用场景，比如：时效性、天气等
- 识别逻辑：通过LLM，判断是否需要联网搜索
- 改写逻辑：通过LLM，对联网搜索的情况，进行查询改写
- 生成策略（可选）：可以提出更详细的平台，以及平台关键词，方便后续进行联网搜索

- 查询类型：
 - 特殊活动安排
 - 限时促销信息
 - 演出时间表
- 处理流程：
 - 判断是否需要联网（置信度>阈值）
 - 对原始查询进行语义改写
 - 生成搜索关键词组合

4. 搜索策略生成 01:08:41

如果后续可以使用 Tavily MCP 进行具体的联网搜索，可以引导LLM生成具体的参数

参数名	类型	是否必填	默认值	说明
query	string	是		搜索关键词
search_depth	string	否		搜索深度，可选值："basic" 或 "advanced"
time_range	string	否		时间范围，如 "day", "week", "month", "year" 或 "all"
max_results	int	否	5	最大返回结果数量
include_images	bool	否	FALSE	是否包含图片
include_answer	bool	否	FALSE	是否包含由 LLM 生成的简短回答
include_raw_html	bool	否	FALSE	是否包含原始 HTML 内容
topic	string	否	"general"	搜索主题，可选值："general" 或 "news"
domains	list	否	-	限定搜索的域名白名单，如 ["bbc.com", "techcrunch.com"]
exclude_domains	list	否	-	排除的域名黑名单

- 核心要素：
 - 主要搜索词：核心关键词（如"上海迪士尼门票"）
 - 扩展搜索词：同义词和相关词（如"票价"、"优惠"）
 - 搜索平台：推荐使用官方站点或特定搜索引擎
 - 时间范围：根据需求指定（最近一周/一个月等）
- 参数规范：
 - 使用JSON格式确保结构化输出
 - 包含必填项（query）和选填项（search_depth等）
- 工具集成：
 - 可调用Tavily等AI搜索引擎
 - 支持通过MCP协议进行标准化调用

五、双向改写01:23:48

1. Query2Doc 01:24:01

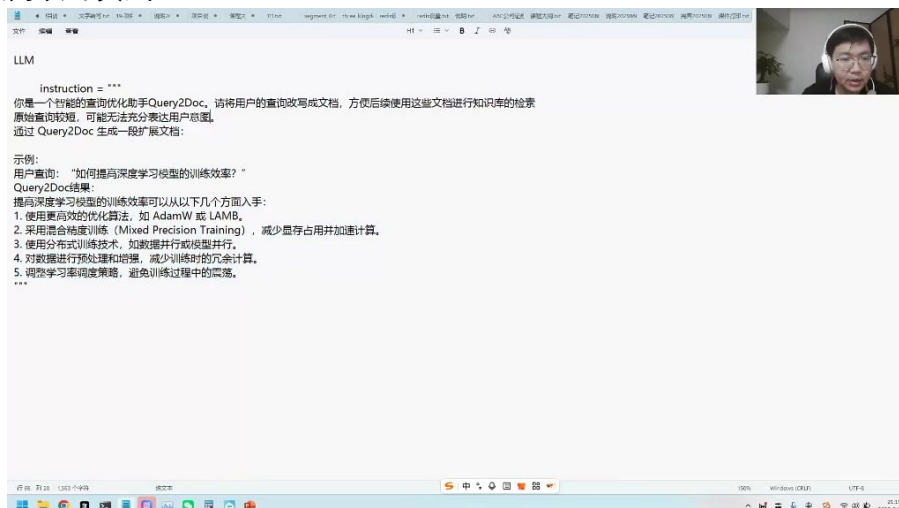
- **核心思想：**将用户简短查询改写成扩展文档，缓解短文本向量化效果差的问题
- **实现方法：**
 - 通过LLM生成包含多个相关维度的扩展文档
 - 示例：用户查询"如何提高深度学习模型的训练效率？"可改写成包含优化算法、混合精度训练、分布式技术等维度的文档
- **应用场景：**当原始查询较短，无法充分表达用户意图时使用
- **注意事项：**
 - 改写结果不是最终答案，而是用于知识库检索的中间产物
 - 计算成本较高，需根据场景价值权衡使用

2. Doc2Query 01:24:24



- **核心思想：**为文档片段生成可能的用户查询，建立文档与问题的多维度联系
- **实现方法：**
 - 针对文档内容预测用户可能提出的各种问题
 - 示例：文档内容涉及深度学习训练优化时，可生成"如何选择优化器？"、"混合精度训练优势？"等问题
- **优势：**
 - 增加文档被检索到的概率
 - 建立问题与文档的深层语义联系
- **典型应用：**知识库预标注，为文档片段添加可能的问题标签

3. 双向改写实践 01:25:07



- **完整流程：**
 - 定义LLM指令模板，明确改写要求
 - 提供改写示例（如深度学习训练效率案例）
 - 执行实际查询改写（如"中国有哪些滑雪胜地"）
 - 使用改写结果检索知识库
- **指令设计要点：**
 - 明确角色设定（智能查询优化助手）
 - 说明改写目的（便于知识库检索）
 - 提供标准示例格式
- **系统集成：**
 - 可作为RAG系统的独立模块
 - 需要与知识库检索、答案生成等模块协同工作
- **成本考量：**
 - 改写会增加计算开销
 - 需根据查询价值决定是否启用改写
 - 对时效性、准确性要求高的场景更适合使用

六、索引扩展01:39:12

1. 离散索引扩展 01:39:22

- **技术实现：**使用关键词抽取、实体识别等技术生成离散索引，与向量检索形成互补
- **典型应用：**BM25算法作为改进版TF-IDF，通过词频饱和度和文档长度归一化提升相关性排序效果

算法公式：

$$BM25(Q,D) = \sum_{i=1}^n DF(q_i) \cdot TF_{BM25}(q_i, D)$$

$$TF_{BM25} = \frac{(k_1 + 1) \cdot tf}{k_1 \cdot (1 - b + b \cdot \frac{dl}{avgdl}) + tf}$$

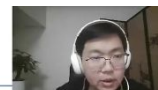
- $DF = \log(\frac{N - n + 0.5}{n + 0.5} + 1)$

2. 连续索引扩展 01:39:36

- **模型组合：**结合多种向量模型（如OpenAI的Ada、智源的BGE）进行多路召回
- **优势特点：**不同模型取长补短，通过embedding计算向量间夹角实现语义匹配

3. 混合索引召回 01:39:48

RAG高效召回方法（索引扩展）



4. 索引扩展

- 1) **离散索引扩展：**使用关键词抽取、实体识别等技术生成离散索引，与向量检索互补，提升召回准确性。
- 2) **连续索引扩展：**结合多种向量模型（如OpenAI的Ada、智源的BGE）进行多路召回，取长补短。
- 3) **混合索引召回：**将BM25等离散索引与向量索引结合，通过Ensemble Retriever实现混合召回，提升召回多样性

BM25是一种经典的文本检索算法，它是TF-IDF（词频-逆文档频率）的改进版本，通过更精细的词频饱和度和文档长度归一化，提升了检索的相关性排序效果。

$$BM25(Q,D) = \sum_{i=1}^n IDF(q_i) \cdot TF_{BM25}(q_i, D)$$

$$TF_{BM25} = \frac{(k_1 + 1) \cdot tf}{k_1 \cdot (1 - b + b \cdot \frac{dl}{avgdl}) + tf}$$

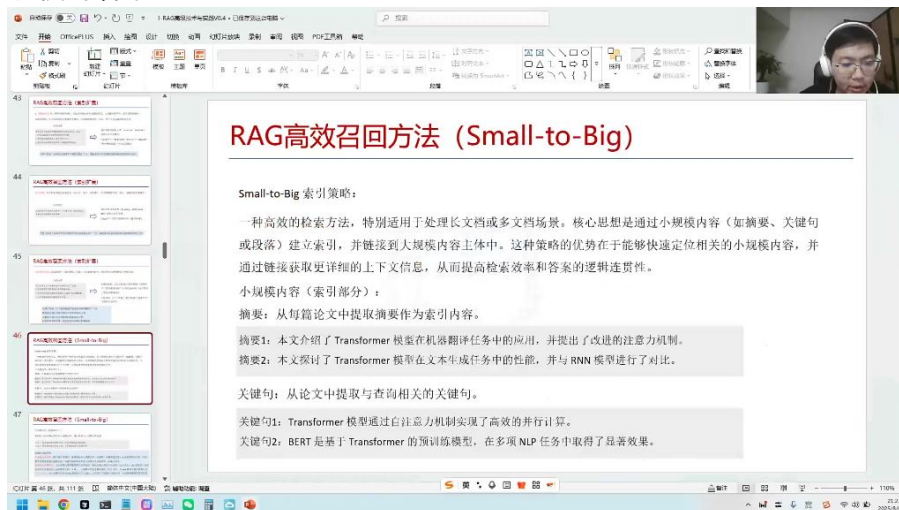
$$IDF = \log\left(\frac{N - n + 0.5}{n + 0.5} + 1\right)$$

- 
- **组合方式：**将BM25等离散索引与向量索引结合，通过Ensemble Retriever实现
- **应用建议：**精确检索场景推荐使用，如文档关键词匹配后再进行向量相似度计算

- **策略选择**：不必全部采用，根据实际需求选择1种或组合多种策略

七、Small-to-Big策略01:40:59

1. 小规模内容索引 01:41:07



RAG高效召回方法 (Small-to-Big)

Small-to-Big 索引策略：

一种高效的检索方法，特别适用于处理长文档或多文档场景。核心思想是通过小规模内容（如摘要、关键词或段落）建立索引，并链接到大规模内容上体中。这种策略的优势在于能够快速定位相关的小规模内容，并通过链接获取更详细的上下文信息，从而提高检索效率和答案的逻辑连贯性。

小规模内容（索引部分）：

摘要：从每篇论文中提取摘要作为索引内容。

摘要1：本文介绍了Transformer模型在机器翻译任务中的应用，并提出了改进的注意力机制。

摘要2：本文探讨了Transformer模型在文本生成任务中的性能，并与RNN模型进行了对比。

关键词：从论文中提取与查询相关的关键词。

关键词1：Transformer模型通过自注意力机制实现了高效的并行计算。

关键词2：BERT是基于Transformer的预训练模型，在多项NLP任务中取得了显著效果。

- **内容类型**：包括摘要（如论文摘要）、关键词（如技术核心语句）或段落
- **生成技术**：通过摘要生成、关键词提取等技术从大规模内容中提取
- **实际案例**：Google Notebook LM工具采用此策略，先匹配摘要再关联全文

2. 大规模内容链接 01:41:40

- **关联方式**：通过文档ID、URL或指针建立预定义链接
- **内容示例**：完整PDF文档（含实验数据）、技术白皮书（含架构细节）
- **优势体现**：保持检索效率的同时可获得完整上下文信息

3. 检索机制 01:42:10

- **两阶段检索**：先在小规模内容中匹配查询，再链接对应大规模内容
- **上下文补充**：将完整文档作为RAG系统输入，提升答案连贯性
- **适用场景**：特别适合长文档（如学术论文）和多文档检索场景
- **实践建议**：类似Notebook LM的"摘要-全文"分层处理模式最有效

八、知识库处理01:44:53

1. 问题生成 01:44:58

场景1：知识库问题生成与检索优化

Thinking：当用户提问与知识切片的相似度不高时，能否通过AI为每个知识切片生成可能的问题，通过问题与问题的匹配来提高检索准确度？

TO DO：对知识库进行问题生成，并进行检索优化

- 自动生成多样化问题：为知识库中的每个知识切片自动生成多种类型、不同难度的问题

`generate_questions_for_chunk()`：为单个知识切片生成基础问题

`generate_diverse_questions()`：生成更多样化的问题（8个）

- 构建双重检索索引：同时构建基于原文内容和生成问题的BM25检索索引
- 检索评估：针对两种检索方式，进行详细的评估

-
- **Doc2Query原理**：当用户提问与知识切片相似度不高时，通过AI为每个知识切片生成可能的问题，通过问题与问题的匹配来提高检索准确度
- **生成方法**：

- generate_questions_for_chunk(): 为单个知识切片生成基础问题
 - generate_diverse_questions(): 生成更多样化的问题 (8个)
 - **应用场景:** 适用于知识库信息未经充分梳理, 用户问题与知识库匹配度不高的情况
2. 特征工程 01:46:09
- **传统特征工程:** 在机器学习阶段对输入模型的特征进行加工处理, 以更好发挥模型能力
 - **现代应用:** 对现有数据进行预处理后再输入大模型进行推理, 与传统特征工程逻辑一致
3. 数据预处理 01:46:39
- **关键作用:** 是知识库处理中的核心环节, 直接影响后续模型推理效果
 - **预处理内容:** 包括知识切片、问题生成、质量评估等步骤

九、知识库问题生成01:53:21

1. 多样化问题生成 01:53:24
- **问题类型:** 包括直接问、间接问、对比问等多种形式
 - **示例:** 针对"上海迪士尼乐园位于浦东新区, 2016年6月16日开园, 面积390公顷"的知识切片, 可生成:
 - 上海迪士尼乐园何时开园?
 - 中国大陆第一座迪士尼在哪里?
 - 与美国迪士尼相比有什么独特之处?
 - 主要乐园类型有哪些?
 - 占地多少公顷?
2. 问题类型分类 01:53:38
- **输出格式:** 采用JSON格式, 包含问题内容、问题类型、难易程度等字段
 - **质量保证:** 通过添加答案验证环节, 确保生成问题与知识文档相关性强
3. 问题质量评估 01:53:47
- **评估方法:** 让AI预先生成问题对应的答案, 验证知识文档能否回答该问题
 - **示例:**
 - 问题: 上海迪士尼乐园哪年开业?
 - 验证答案: 2016年 (与文档一致)
 - 问题: 迪士尼小镇属于哪个部分?
 - 验证答案: 需检查文档是否包含此信息

十、检索优化01:57:26

1. BM25算法 01:57:29
- **检索策略:** 采用关键词匹配方法进行问题检索
 - **双重索引:** 同时构建基于原文内容和生成问题的BM25检索索引
2. 检索准确率对比 01:57:56
- **对比实验:**
 - 原文检索: 直接匹配用户问题与知识文档
 - 问题检索: 匹配用户问题与生成的问题集
 - **结果示例:**
 - "体验最刺激过山车去哪个区域": 问题检索匹配成功, 原文检索失败
 - "可以带食物吗": 两种方法均匹配成功
 - "什么时候人气好": 问题检索匹配更准确
3. 检索结果分析 01:59:21
- **优势场景:** 问题检索在用户提问与知识文档表述差异大时表现更优
 - **关键因素:** 检索效果取决于生成问题的质量和多样性

十一、对话知识抽取 02:01:30

1. 信息提取方法 02:01:33

- **提取内容**：从用户对话中抽取事实性信息、用户需求、常见问题、操作流程、注意事项等
- **输出格式**：包含知识类型、内容、置信度、来源、关键词和具体分类

2. 知识整合 02:01:53

- **应用场景**：产品上线后从日常对话中持续丰富知识库
- **工具函数**：从conversation中抽取信息()、批量抽取合并知识点()

3. 知识分类 02:02:22

- **分类维度**：
 - 知识类型（事实性、流程性等）
 - 置信度（信息可靠程度）
 - 来源（原始对话来源）
 - 关键词（便于检索）
 - 具体分类（业务维度分类）

十二、RAG高级技术与实践 02:02:51

1. 知识库问题生成

1) 条件问题示例 02:03:04

场景1：知识库问题生成与检索优化

5. 为什么说上海迪士尼乐园在中国具有特殊意义？
类型：推理问，难度：困难，角度：历史与文化价值，能否回答：True，回答的答案：因为它是中国大陆首座迪士尼主题乐园，标志着迪士尼品牌在中国市场的落地与扩展

6. 上海迪士尼乐园占地多少公顷？
类型：直接问，难度：简单，角度：面积数据，能否回答：True，回答的答案：390公顷

7. 迪士尼小镇属于上海迪士尼乐园的哪个部分？
类型：间接问，难度：中等，角度：结构组成，能否回答：True，回答的答案：迪士尼小镇是上海迪士尼乐园的一部分，与七大主题园区并列

8. 若要全面游览上海迪士尼乐园的全部主题园区，需要了解哪些区域？
类型：条件问，难度：困难，角度：规划与认知，能否回答：True，回答的答案：需要了解米奇大街、奇想花园、探险岛、宝藏湾、明日世界、梦幻世界和迪士尼小镇这七大主题园区

1-知识库问题生成与检索优化-BM25.py

- **推理问示例**："为什么说上海迪士尼乐园在中国具有特殊意义？" 属于历史与文化价值角度，难度为困难
- **直接问示例**："上海迪士尼乐园占地多少公顷？" 属于面积数据角度，难度为简单
- **间接问示例**："迪士尼小镇属于上海迪士尼乐园的哪个部分？" 属于结构组成角度，难度为中等
- **条件问示例**："若要全面游览上海迪士尼乐园的全部主题园区，需要了解哪些区域？" 属于规划与认知角度，难度为困难

2. 对话知识沉淀 02:03:15

1) 单次对话提取 02:03:19

场景2：对话知识沉淀



示例1: 从单次对话中提取知识

对话内容:

用户: "我想去上海迪士尼乐园玩, 门票多少钱?"

AI: "上海迪士尼乐园的门票价格根据日期有所不同。平日成人票价为399元, 周末和节假日为499元。儿童票 (1.0-1.4米) 平日为299元, 周末为374元。1.0米以下儿童免费。"

用户: "需要提前预订吗?"

AI: "建议提前预订, 特别是周末和节假日, 可以通过官方网站或第三方平台预订。"

用户: "从浦东机场怎么去?"

AI: "从浦东机场到迪士尼乐园可以乘坐地铁2号线到广兰路站, 然后换乘11号线到迪士尼站, 全程约1小时。也可以打车, 约40分钟。"



2-对话知识沉淀.py

- 提取方法: 从用户与AI的多轮对话中提取知识点, 如门票价格、预订建议、交通方式等
- 提取数量: 示例中从多次对话中提取了22个知识点, 每个知识点都是独立的知识片段

2) 知识分类方法 02:03:35

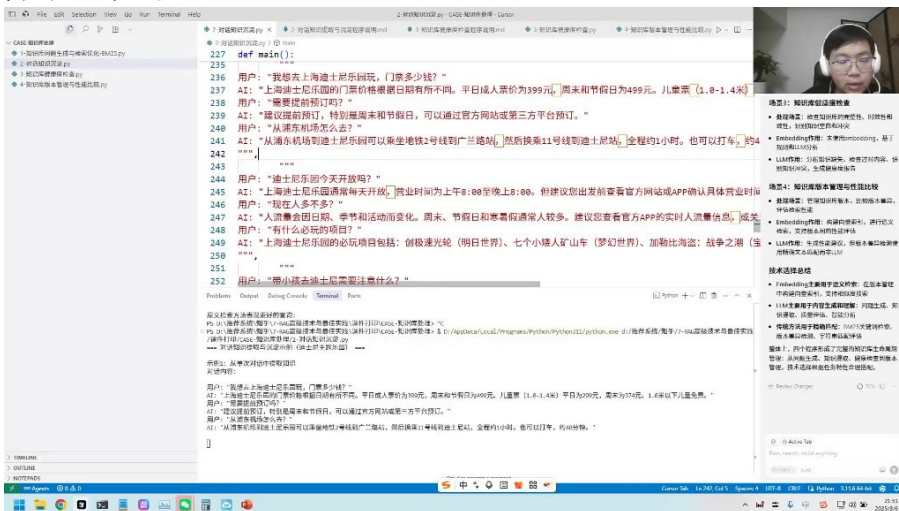
- 事实类型: 如门票价格、开放时间、表演节目、停车场收费标准等具体信息
- 需求类型: 用户表达的需求信息, 如"我想去上海迪士尼", 这类信息需要过滤掉
- 流程类型: 主要涉及交通情况等操作流程信息
- 注意事项类型: 如建议提前预定门票等提醒类信息

3) 知识合并策略 02:03:51

- 过滤原则: 首先过滤掉需求和问题类型的知识, 这些属于用户提问而非答案
- 合并方法: 使用大模型将同类型知识智能合并, 保留重要信息, 去除重复内容
- 合并效果: 可将原本十几个字的短知识点合并为100多字的丰富知识描述
- 事实类合并示例: 将门票价格、开放时间、表演节目等信息合并为完整的事实描述
- 流程类合并: 将不同交通方式信息整合为完整的出行指南

3. 知识库健康检查 02:06:30

1) 冲突知识检测 02:06:36



- Embedding作用: 主要用于语义检索, 在版本管理中构建向量索引支持相似度搜索
- LLM作用: 主要用于内容生成和理解, 包括问题生成、知识提取、质量评估等
- 传统方法应用: BM25关键词检索、版本差异检测、字符串匹配等精确匹配场景
- 完整生命周期: 包含问题生成、知识提取、健康检查和版本管理四个完整环节

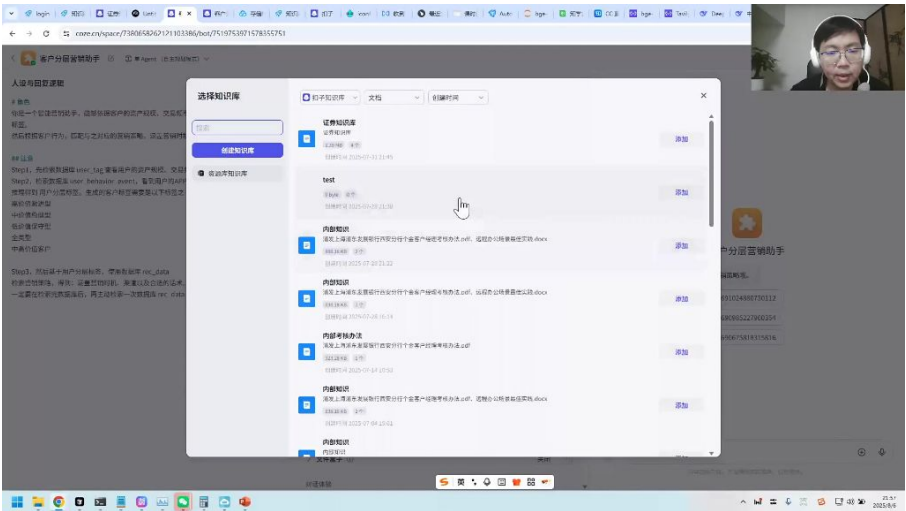
4. 知识库版本管理 02:10:45

1) 性能评估指标 02:10:58

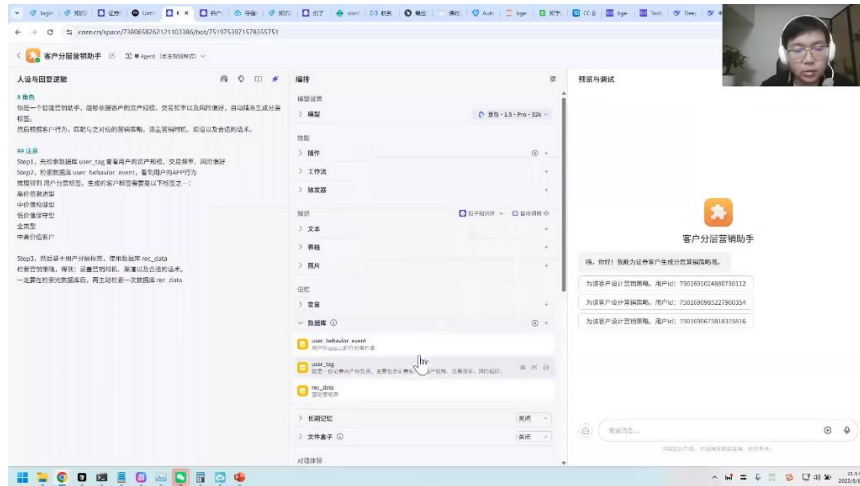
- **健康检查维度：**通过三大检测维度评估知识库质量，包括缺少知识检测（如未覆盖"特别活动"和"停车费"查询）、过期知识检测（基于时间范围判断，例如2025年8月6日检测到5月1日-5日的政策已过期）、冲突知识检测（版本差异导致的内容矛盾）
- **检测方法：**
 - **关键词匹配法：**通过测试问题与答案关键词的包含关系判断知识完整性（如"上海迪士尼乐园在哪里"需包含"浦东新区"）
 - **时间戳比对：**传入当前时间与知识有效期进行新鲜度打分（检测到价格信息和活动信息已过期案例）
 - **一致性专家模型：**大模型扮演冲突检测角色，识别版本差异（如两个活动时间描述冲突）
- **版本差异分析：**
 - **量化对比：**版本1含3条知识，版本2扩展至5条，新增2条并优化原有内容
 - **准确率提升：**版本1回答准确率60%（3/5），版本2达100%（5/5）
 - **响应时间：**知识增加可能导致检索时间轻微上升
- **回归测试要求：**需验证旧版本问题在新版本中的通过率（案例显示5个历史问题全部通过）

5. 企业知识库实践 02:12:24

1) 文件夹式管理 02:12:39



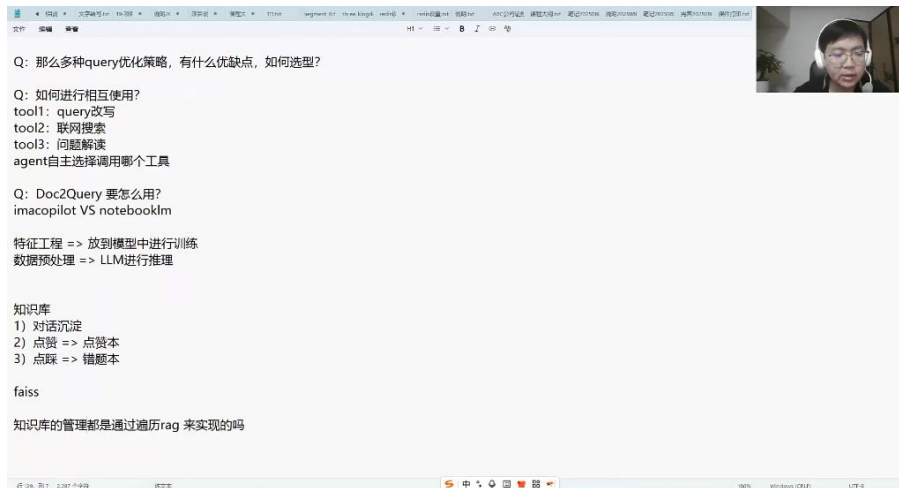
- **多库并存特点：**
 - **分类存储：**企业通常按业务领域划分多个知识库（如证券知识、内部制度、考核办法等）
 - **扁平化结构：**单个知识库采用浅层文件夹结构，类似文件目录管理
 - **部门级应用：**单个部门内部也可能存在多个专用知识库（如营销策略库、客户标签库）



- 典型应用案例：
 - 客户分层系统：通过三阶段流程实现精准营销（资产规模分析→行为事件匹配→策略话术生成）
 - 动态检索机制：需二次检索rec_data确保策略时效性（案例用户ID：7501691024880730112等）
 - 标签体系：包含"高价值激进型"、"中价值稳健型"等5类标准标签

6. 技术实现方式 02:19:38

1) 高代码开发 02:19:51



- 开发平台选择：可使用腾讯智能体开发平台等标准化工具，也支持企业自主开发个性化解决方案
- 特征工程流程：数据预处理⇒特征工程⇒放入模型训练⇒LLM推理
- 知识管理工具：包含点赞本（优质内容收集）、错题本（问题修正记录）等功能模块

7. 知识库与RAG区别 02:24:51

1) 知识库与RAG的共性 02:24:54

- 共同基础：都是对知识库的应用，属于知识管理系统的不同环节
- 核心维度：包含问题生成、知识沉淀、健康检查、版本性能比较四大管理主题

2) 知识库的定义与原始数据 02:25:18

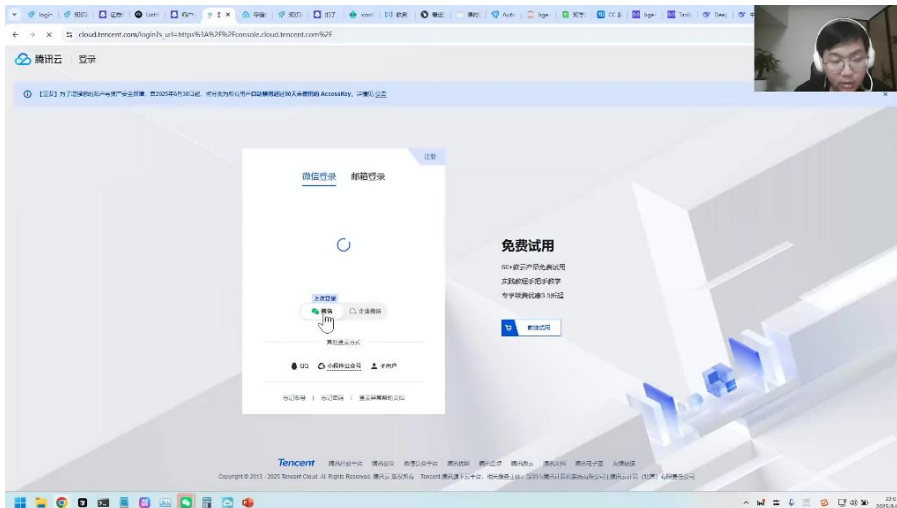
- 本质特征：指未经深度处理的原始文档集合，如企业内部原始资料
- 处理阶段：文档需经过"喂入"知识库的预处理过程才能使用

3) RAG作为知识库的使用过程 02:25:29

- 工作流程：
 - 用户提问查询

- 知识检索与问题完善 (A阶段)
- LLM生成最终回答 (B阶段)
- **技术特点**: 实现知识检索 (Retrieval) 与生成 (Generation) 的增强 (Augmented) 组合

4) 企业中知识库的开发模式 02:25:59



- **标准化方案**:
 - 使用大厂产品如腾讯智能体开发平台、COS等
 - 提供文档导入、问答生成等基础功能
 - 通常按年付费，三年后可买断
- **自主开发**:
 - 解决个性化需求
 - 需实现错题本等高级功能
 - 典型案例包含智能体和知识库两大模块

5) 知识库与向量数据库的关系 02:26:47

- **功能分工**: 知识库存储原始文档，向量数据库负责知识筛选
- **数学基础**: 使用余弦相似度 ($\cos\theta$) 等算法进行向量计算

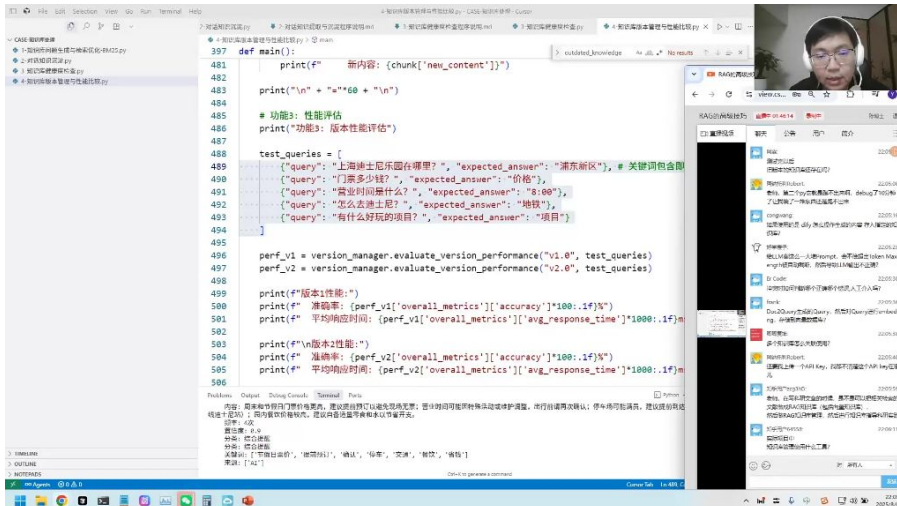
6) 向量数据库的作用与价值 02:26:56

- **核心优势**: 能快速处理海量数据的相似度计算
- **典型应用**: FAISS等工具实现高效语义检索

7) 知识库与向量数据库在筛选知识中的应用 02:27:36

- **协作流程**: 向量数据库通过数学计算识别最优知识片段，供知识库调用
- **评估指标**: 包含准确率 (如浦东新区关键词匹配)、响应时间等性能参数

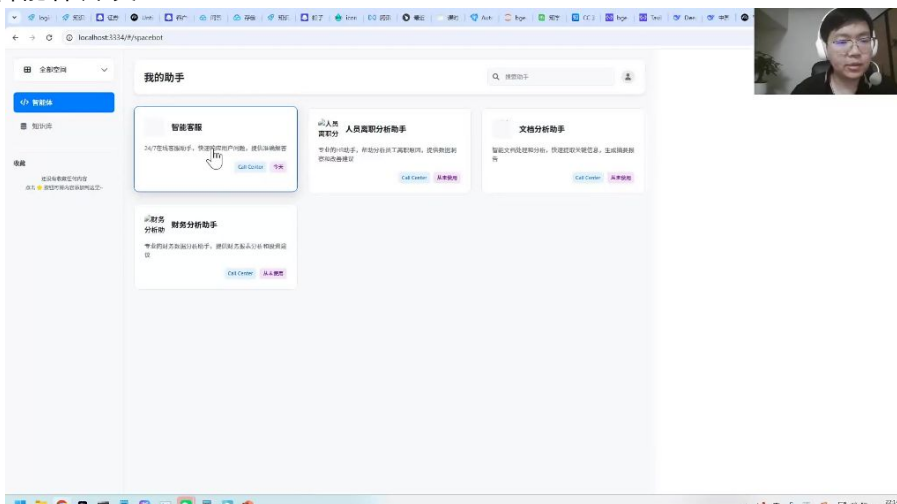
8) 企业自主开发知识库的案例概述 02:27:57



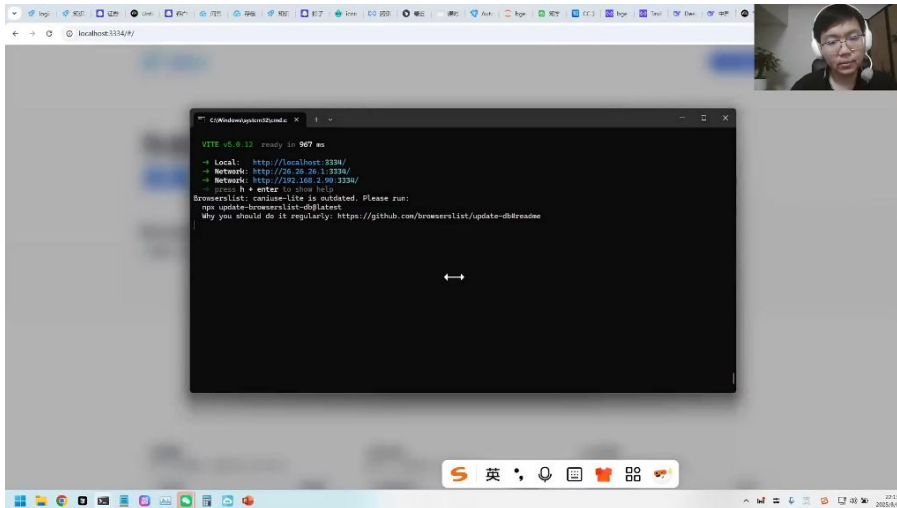
- 开发要点：
 - 需整合企业特定功能需求
 - 典型评估方法：设置测试问题集（如"上海迪士尼位置"对应"浦东新区"关键词）
 - 准确率计算：采用关键词包含策略或LLM判断
- 版本管理：支持v1.0/v2.0等多版本性能对比（准确率、响应时间等指标）

8. 本地开发案例 02:28:31

1) 智能体开发 02:29:00



- 开发模式：采用独立开发平台模式，前端使用Vue框架，后端使用Python进行封装衔接
 - 功能模块：
 - 智能体管理界面
 - 知识库管理界面
 - 24/7在线客服助手
 - 文档分析助手
 - HR离职分析助手
 - 开发流程：通过客户访谈梳理需求→生成测试问题→搭建流程→提取共性逻辑
- ### 2) 知识库管理 02:29:10

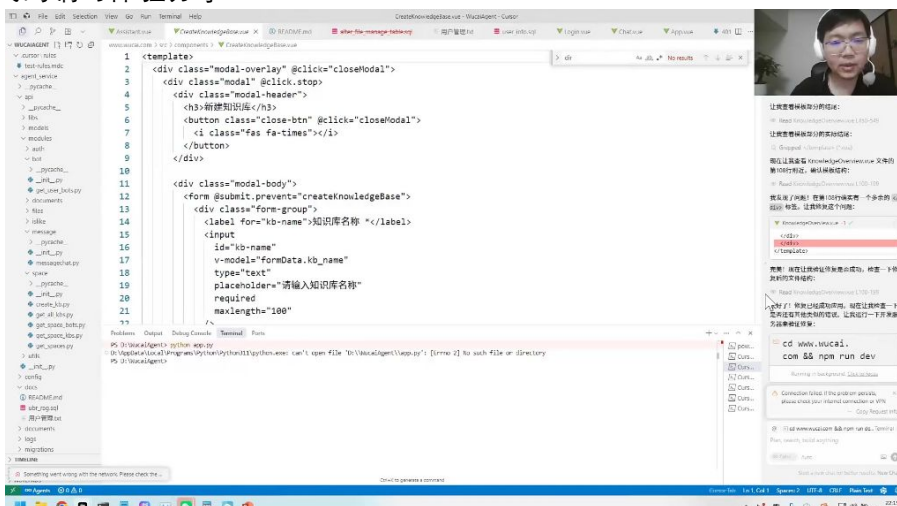


- 管理方式：
 - 支持知识上传和查看
 - 对话内容沉淀功能
 - 点赞生成点赞本
 - 点踩生成错题本
- 技术选型：
 - 使用FAISS进行向量检索
 - 可通过Agent开发平台(如Coze)管理
 - 纯文本占比约10%

3) 高代码与低代码开发模式 02:30:36

- 低代码特点：
 - 通过可视化拖拽完成开发
 - 提供标准化产品
 - 代表平台：Coze、腾讯智能体平台
- 高代码特点：
 - 需要手动编写大量代码
 - 提供个性化定制产品
 - 使用Cursor等专业开发工具

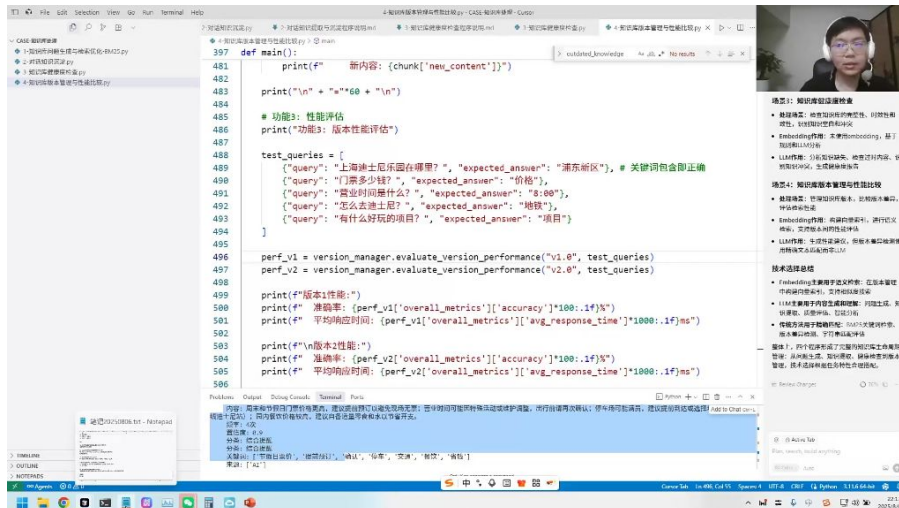
4) 代码编写体验分享 02:31:12



- 开发体验：
 - 单个任务平均耗时约20分钟
 - 需要反复测试和修正

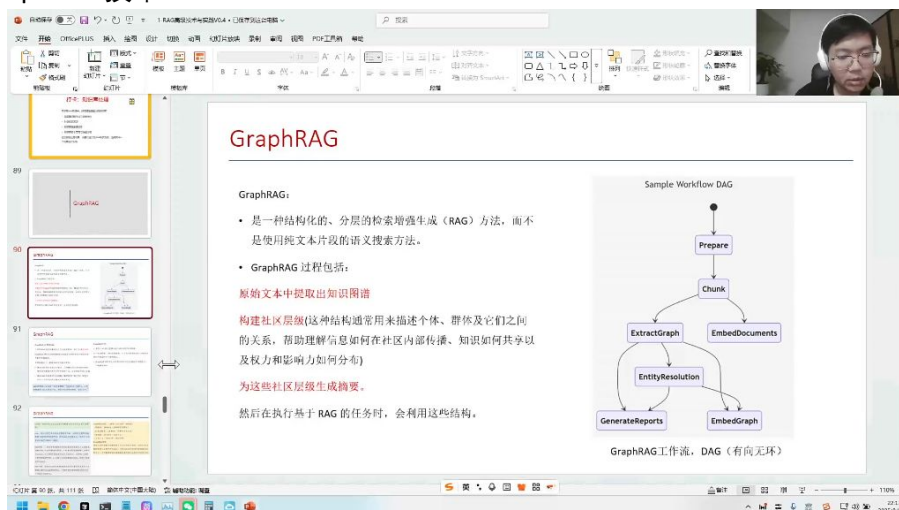
- 网络延迟会影响开发效率
- 价值体现：
 - 完成产品后成就感显著
 - 可直接交付客户使用
 - 能实现更个性化的功能需求

5) 高代码与低代码的定义 02:32:26



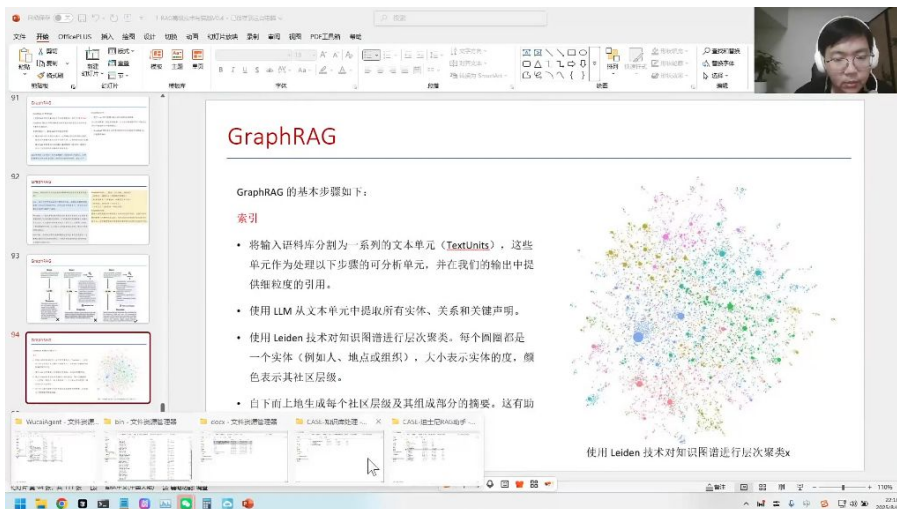
- 核心区别：
 - 低代码：无代码可见，通过配置完成
 - 高代码：需要编写完整代码逻辑
- 适用场景：
 - 低代码适合标准化需求
 - 高代码适合个性化定制
- 预期管理：
 - 两种模式实现的最终效果存在差异
 - 需要根据客户需求选择合适的开发方式

9. GraphRAG技术 02:32:51



- 结构化方法：采用结构化的、分层的检索增强生成(RAG)方法，区别于传统纯文本片段的语义搜索方法
- 核心优势：通过知识图谱处理能提供更高质量的答案，因为对原始文档进行了特殊工程处理

1) 知识图谱应用 02:33:00



- 处理流程:
 - 将输入语料库分割为文本单元(TextUnits)
 - 使用LLM提取实体、关系和关键声明
 - 应用Leiden技术进行知识图谱层次聚类
 - 自下而上生成社区层级摘要
- 可视化特征: 聚类结果中圆圈表示实体(人/地点/组织), 大小表示实体度, 颜色表示社区层级

2) 社区层级抽取 02:33:14

- 典型应用: 以三国演义为例, 系统会进行以下预处理:

- 实体抽取(人物/地点/组织)
- 关系识别
- 社区划分(如魏蜀吴阵营)

- 中间产物:

- 实体(entity)列表
- 摘要(summarized)信息
- 实体关系图谱
- 社区(communities)结构

3) 开源工具使用 02:34:06

GraphRAG

GraphRAG方法:

<https://github.com/microsoft/graphrag>

Step1, 下载源代码

git clone <https://github.com/microsoft/graphrag>

cd graphrag

Step2, 下载依赖并初始化项目

pip install poetry

poetry install

```
Creating Virtualenv graphrag-32177.40-py3.11 in C:\Users\cheny\AppData\Local\
Installing dependencies from lock file
Package operations: 480 installs, 1 update, 0 removals
- Installing attrs (24.2.0)
- Installing zipp (3.20.0)
- Installing referencing (0.35.1)
- Installing six (1.16.0)
- Installing jsonschema-specifications (2024.10.1)
- Installing platformdirs (4.3.6)
- Installing python-dateutil (2.9.0.post0)
- Installing pywin32 (309)
- Installing traitlets (5.14.3)
- Installing types-python-dateutil (2.9.0.20241003)
- Installing arrow (1.3.0)
- Installing fastjsonschema (2.20.0)
- Installing jsonschema (4.23.0)
- Installing jupyter-core (5.7.2)
- Installing pycparser (2.22)
- Installing pyzmq (26.2.0)
- Installing tomli (2.0.1)
- Installing cffi (1.17.1)
- Installing fqdn (1.5.1)
- Installing idna (3.10)
- Installing isoduration (20.11.0)
- Installing jsonpointer (3.0.0)
- Installing jupyter-client (8.6.3)
- Installing markupsafe (3.0.1)
```

- 工具来源: 微软开源项目(microsoft/graphrag)
- 安装步骤:
 - git clone <https://github.com/microsoft/graphrag>
 - cd graphrag

- 

初始化

```
poetry run poe index --init --root .
```

正确运行后，此处会在graphrag目录下生成output、prompts、.env、settings.yaml文件

Step3, 将待检索的文档放到 ./input 目录下

- three kingdoms

Step4, 修改配置文件

对 .env 文件配置 api key

GRAPHFRAG API KEY=sk-T2DgphgTxxPn-nddAa-igT1B2MvF2D4MC2T0K1JH1y7P0dW24

修改 settings.yaml, 设置 model 为 gpt-4o-mini 减少成本

```

3 encoding_model: C1000k_base
4 # skip_pure_funcs: {}
5 llm:
6   api_key: "${GRABRAG_API_KEY}"
7   type: OpenAIChat # or azure_openai_chat
8   model_gpt4o: gpt-4o
9   model_supports_json: true # recommended if this is true
10  max_tokens: 1000
11  request_timeout: 100s
12  url_base: https://(instance).openai.azure.com
13  api_version: 2024-02-15-preview
14  organization: organization.id
15  deployment_name: azure_model_deployment_name
16  # max_retry_attempts: 1 # 100000 is set a leaky bucket
17  requests_per_minute: 10000 # set a leaky bucket
18  max_retries: 10
19  max_retry_wait: 10s
20  # check_rate_limit_recommendation: true # whether
21  # concurrent_requests: 25 # the number of parallel
22  # temperature: 0 # temperature for sampling
23  # top_p: 1 # top-p sampling
24  # top_k: 1 # top-k sampling
25  # num_repetitions: 1 # number of repetitions to generate

```

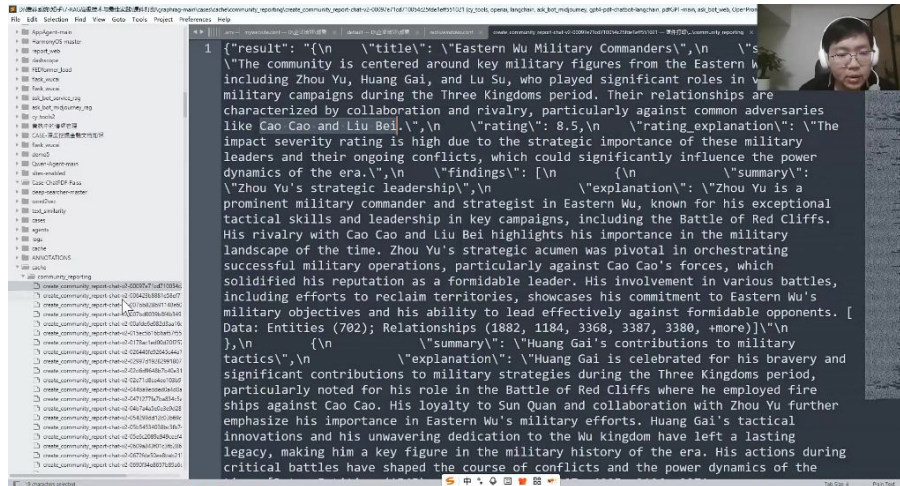
-

1. GraphRAG索引创建

[illegible]

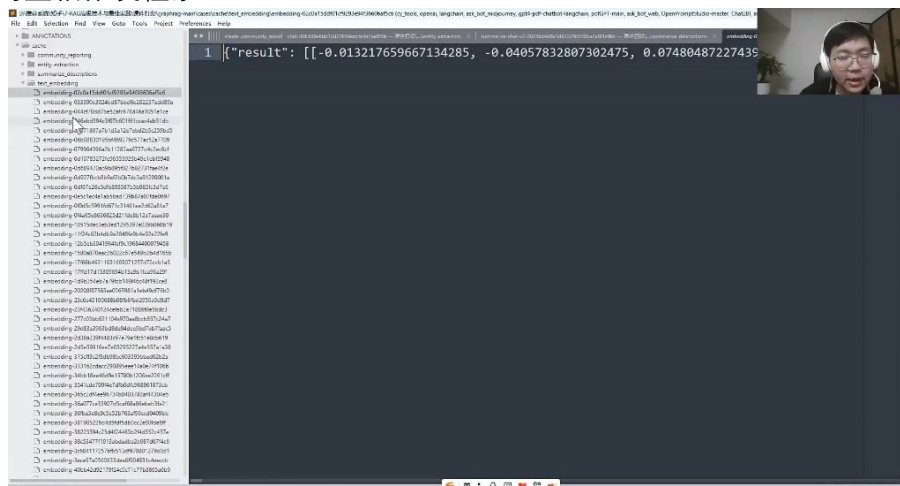
- **处理流程**：通过python -m graphrag.index命令创建索引，包含文本单元生成、实体提取、实体汇总、实体图构建等步骤
- **耗时特征**：预处理耗时约30分钟，处理成本约100元（使用GPT-4o-mini模型）
- **输出结构**：在./cache目录下生成4个文件夹
(community reporting/entity extraction/summarize descriptions/text embedding)

1) 三国演义案例展示 02:35:34



- 实体提取：系统自动识别关键实体如"三国演义(事件)"、"罗贯中(人物)"、"长江(地理)"等
- 关系标注：建立实体间关系网络，如"曹操→敌对→袁绍"等关系
- 社区报告：生成21558行×15列的实体关系矩阵，包含类型、坐标等元数据

2) 向量相似度检索 02:37:22



- 向量存储：每个实体存储768维嵌入向量（如[0.01047, 0.02297,...]）
- 检索原理：通过余弦相似度计算实体间关联度
- 应用场景：支持"天水城→姜维"等实体关系的快速检索

3) 预处理耗时说明 02:37:32

- 性能瓶颈：文本规模直接影响处理时间，万级实体需要30+分钟
- 成本构成：主要来自大模型API调用（GPT-4o-mini生成摘要和关系）

2. GraphRAG查询模式 02:38:11

1) 全局查询模式 02:38:17

5. 康熙朝与乾隆朝

- 1. **背景**：康熙朝是清朝的上升期，继承了明朝的权力，并在其父去世后成为康熙的第一位皇帝。父子关系在国家的建立和发展中起到了关键作用 [Data: Reports (42, 115)].
- 2. **康熙朝**：康熙的统治被称为“康乾盛世”，以其卓越军事才能著称，参与了多次重要战役，支持清朝的军事扩张 [Data: Reports (212, 79, 336, 396)].
- 3. **胤禩/胤禔**：另一位重要的皇子，他在康熙的御前下旨与多次重要战役，展示了他在军事事件中的影响力 [Data: Reports (178, 153)].
- 4. **胤禩/胤禔**：作为康熙的军事顾问，帮助其战略决策和军事部署，对康熙的决策产生了深远影响 [Data: Reports (195, 42)].
- 5. **关系**：虽然胤禩与胤禔关系复杂，但胤禔与胤禔的关系复杂，涉及皇权和内心冲突 [Data: Reports (249, 180)].

其他相关人物

- 1. **胤禩/胤禔**：胤禔与胤禔关系复杂，但胤禔与胤禔的关系复杂，涉及皇权和内心冲突 [Data: Reports (378, 160)].
- 2. **胤禩/胤禔**：胤禔的主要战略家，与胤禔的对抗在清朝历史中占据了重要地位 [Data: Reports (249, 180)].
- 3. **胤禩/胤禔**：著名的皇帝胤禔，显示了胤禔在政治环境中的重要性 [Data: Reports (313)].

结论

胤禔的历史地位与胤禔的众多人物密切相关。这些人物之间的冲突和互动不仅塑造了胤禔的军事和政治生涯，也在清朝历史中发挥了重要作用。通过对胤禔的深入研究，我们可以更好地理解胤禔的统治。

- 

3. 刘敬

刘敬是曹魏的重要谋士，以其卓越的智谋和战略眼光而闻名。他在曹魏的军事和政治决策中提供了关键性的建议，帮助曹魏在复杂的局势中找到明智的选择 [Data: Entities (90); Relationships (未提供)]。

4. 刘备

刘备是曹魏的主要对手之一，最初与曹魏有合作，但后因追求权力斗争而成为敌人。刘备的崛起对曹魏构成了直接威胁，二者之间的竞争深刻影响了三国时期的历史进程 [Data: Entities (2); Relationships (497, 256, more)]。

5. 吕布

吕布是另一位与曹魏有过激烈冲突的著名武将。他以勇猛著称，但因其反复无常的性格而受到警惕。他的行为不仅影响了曹操和曹魏之间的发展，还加剧了当时的内乱 [Data: Entities (未提供); Relationships (3789, more)]。

总结

曹魏的周围有许多重要人物，他们在不同的历史阶段对曹魏的军事和政治生涯产生了深远的影响。这些人不仅包括曹操的智者和谋士，还包括他的对手。他们的互动和冲突共同塑造了三国时期的历史画卷。

- #### 4. 查询案例演示 02:41:58

- 70 —

python -m graphrag.query --root ./cases --method local "关羽战胜过哪些武将?"

```
python -m graphrag.query --root ./cases --method global "关羽战  
胜过哪些武将?"
```

SPICE55: Global Search Response:

*** 关羽的战绩记录 ***

关羽，字云长，是三国时期著名的将领，以其卓越的武艺和战斗能力而闻名。他与多位敌将有过激烈的战斗，取得了重要的胜利。这些胜利不仅巩固了他的英雄地位，也为他赢得了广泛的尊敬。

*** 关羽的战斗经历 ***

1. **斩杀颜良**：关羽在著名的战斗中斩杀了袁绍的部将，显示了他作为顶级武将的实力。这场胜利奠定了关羽的威名。 [Data: Reports (299, 101, <more>)]

2. **曹操的投降**：关羽在战斗中的多位胜利，包括斩颜良、诛文丑等。这些胜利不仅展示了他的个人实力，也在三国时期的权力斗争中产生了深远的影响。 [Data: Reports (132, 154, 277, 342, <more>)]

3. **水淹七军**：关羽与曹操的交战中也表现出色，进一步巩固了他的声誉。 [Data: Reports (298, 181, <more>)]

4. **斩杀于禁**：关羽在著名的战斗中取得了胜利，展示了他战斗中的卓越才能。 [Data: Reports (323, 255, <more>)]

5. **周仓自刭**：虽然周仓的战斗力和战绩尚未得到证实，但关羽多次交锋中展现了强大的战斗力，最终导致了周仓的死亡。 [Data: Reports (79, 212, 249, 400, <more>)]

*** 战斗的胜利 ***

关羽的胜利不仅提升了他的个人声望，也为蜀汉的军事事业做出了重要贡献。他在战斗中的卓越表现，使他成为三国时期最杰出的武将之一。他的胜利不仅巩固了他的地位，也为他赢得了广泛的尊敬。

关羽是三国时期最著名的武将之一，他的胜利不仅巩固了他的地位，也为他赢得了广泛的尊敬。他的胜利不仅巩固了他的地位，也为他赢得了广泛的尊敬。

- ## 5. 应用场景建议 02:42:25

- 适用标准：高价值文本

- 典型场景：上市公司关联网络分析
- 成本考量：需评估ROI，建议文档价值>\$500时采用

1. GraphRAG介绍

GraphRAG查询

Thinking: 如何让匹配上的entities和关系更多一些?

```
local_search:
# text_unit_prop: 0.5
# community_prop: 0.1
# conversation_history_max_turns: 5
# top_k_mapped_entities: 10
# top_k_relationships: 10
# llm_temperature: 0 # temperature for sampling
# lim_top_p: 1 # top-p sampling
# llm_n: 1 # Number of completions to generate
max_tokens: 12000
```

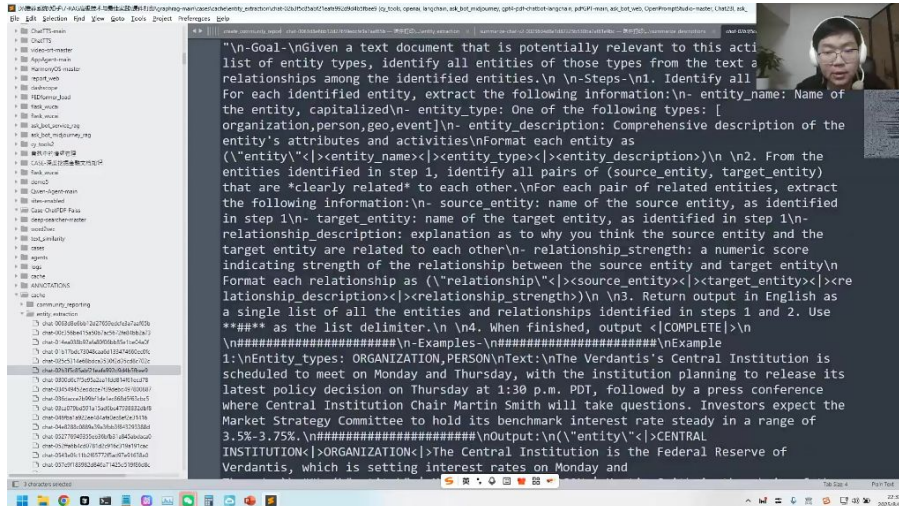
GRAPHIRAG_LOCAL_SEARCH_COMMUNITY_PROP	下文窗口用于社区报告的比例。默认值: 0.1
GRAPHIRAG_LOCAL_SEARCH_CONVERSATION_HISTORY_MAX_TURNS	历史对话中上下文的最大轮次数。默认值: 5
GRAPHIRAG_LOCAL_SEARCH_TOP_K_ENTITIES	从实体链中提取入参中最相关的实体数量。默认值: 10
GRAPHIRAG_LOCAL_SEARCH_TOP_K_RELATIONSIPS	从关系链中提取入参中最相关的关系数量。默认值: 10
GRAPHIRAG_LOCAL_SEARCH_MAX_TOKENS	限制每个模型输出token的数量(如果使用的模型是令牌限制的)。默认值可能为5000。默认值: 12000
GRAPHIRAG_LOCAL_SEARCH_LLM_MAX_TOKENS	限制模型的输出长度;限制输入;更改(如果你有使用令牌限制模型)。默认值可能为1000-1500。默认值: 2000
GRAPHIRAG_GLOBAL_SEARCH_MAX_TOKENS	限制全局的搜索窗口已经进行更改(如果你使用的是令牌限制的模型)。默认值可能为5000。默认值: 12000
GRAPHIRAG_GLOBAL_SEARCH_DATA_MAX_TOKENS	限制的数据窗口大小已经进行更改(如果你使用的是令牌限制的模型)。默认值可能为5000。默认值: 10000
GRAPHIRAG_LOCAL_SEARCH_MAX_TOKENS	默认值: 500
GRAPHIRAG_GLOBAL_SEARCH_REFINE_MAX_TOKENS	限制每次迭代中的窗口进行更改(如果你使用的是令牌限制的模型)。默认值可能为1000-1500。默认值: 2000
GRAPHIRAG_GLOBAL_SEARCH_CONCURRENCY	默认值: 32

- ## 2. GraphRAG应用场景 02:43:15

1) 金融领域应用 02:43:31

- **适用性:** 在金融领域特别有价值，因其需要处理大量实体关系
- **成本考量:** 其他领域应用需权衡成本效益，预处理一本书约需几十元，每个问题1-2元

2) 知识图谱构建 02:44:13



● 组成要素:

- 节点(Node): 代表实体(如人物"张飞"、地点"徐州")
- 边(Edge): 代表实体间关系(如"兄弟关系")
- 属性: 描述实体特征

- **处理流程:** 对原始文档进行二次处理，提取实体、关系、社区报告等信息

3. GraphRAG数据处理 02:44:55

- **预处理工作:** 需要大量前期处理，包括实体定义、关系提取、社区报告整理
- **数据整理:** 对文章中的实体、关系、社区报告进行系统化整理

4. GraphRAG问答质量 02:45:34

- **优势:** 通过结构化处理使回答更准确，信息利用率更高
- **局限性:** 不能完全避免大模型的幻觉问题，但能显著提升知识匹配度

5. 知识库与知识图谱区别 02:46:45

- **知识库:** 原始文档集合(如三国演义原文)
- **知识图谱:** 对原始文档的笔记整理，构建图结构(node,edge)

6. GraphRAG替代方案 02:46:55

- **KAG:** 阿里开发的替代方案
- **传统RAG:** 如Dify、Coze等平台提供的方案，成本更低

7. 千本Agent方法 02:47:21

- **特点:** 对数据做切片处理，使用全量信息回答
- **对比:** 传统RAG基于top5片段，信息可能不全

8. GraphRAG开源工具 02:47:46

- **可用性:** 开源代码工具，基于图理论构建数据模型
- **检索方式:** 可在key tap上检索相关实现

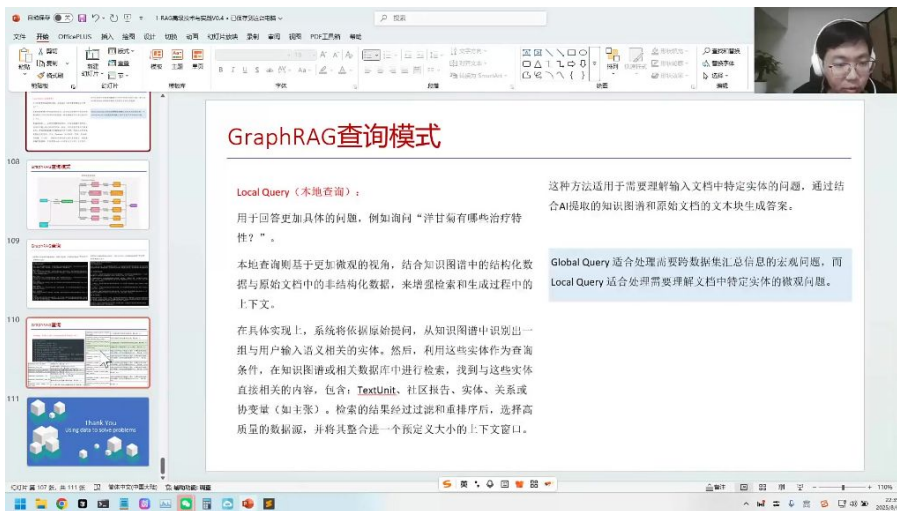
9. 知识存储策略 02:48:46

- **问题存储:** 可将生成的question与原始文本一起存入知识库
- **改写技巧:** 通过API对原始信息增加理解性内容

10. GraphRAG幻觉问题 02:49:46

- **改善程度:** 能提升知识利用率，但不能100%避免幻觉
- **优势:** 更宏观、完整地利用现有知识

11. 查询方法选择 02:50:22



- - **Global Query:** 适合跨数据集汇总信息的宏观问题
 - **Local Query:** 适合理解文档特定实体的微观问题
12. 向量查询原理 02:51:26
- 作用: 筛选chunk知识, 通过数学工具(如cos相似度)计算
 - 流程: 问题+筛选出的chunks→LLM生成回答
13. LangChain封装实现 02:53:50
- **MultiQueryRetriever:** LangChain中的检索封装
 - 使用场景: 更适合传统RAG实现, GraphRAG需要更多自定义
14. RAG技术总结 02:54:58
- 1) Query改写技巧 02:55:10
- 必要性: 用户提问与知识库陈述存在表达差异
 - 方法: 意图识别、问题扩展、陈述句转换
- 2) 联网搜索策略 02:55:31
- 应用场景: 问题需要实时外部信息时
 - 实现: 生成搜索策略并整合结果
- 3) 高效召回方法 02:55:45
- **Small to Big:** 从小范围逐步扩大检索
 - 双向检索: query→doc和doc→query结合
 - 混合检索: 结合多种检索策略
15. RAG与知识库关系 02:57:39
- **RAG:** 知识使用过程(检索→增强→生成)
 - 知识库: 原始数据管理
 - 类比: 知识库如藏书, RAG如回答读者问题
16. 向量数据库作用 02:59:10
- 核心功能: 相似度计算引擎
 - 替代方案: 可用纯关键词检索(非语义检索)

十五、知识小结

知识点	核心内容	实践技巧/易混淆点	重要性
RAG基础概念	检索增强生成技术原理: 数据存储(index)→文档切分(chunk)→向量化→相似度计算→答案生成	chunk size通常300-1000之间(默认800), 300-500 颗粒度更细	★★★★★

查询改写策略	5种改写类型：上下文依赖型/对比型/模糊指代型/多意图型/反问型	需通过prompt工程实现意图识别， JSON格式输出改写结果+置信度	★★★★
混合检索方法	结合关键词检索+向量检索的多通路召回策略	先用简单策略快速召回（如关键词），再用复杂模型重排（如BGE Reranker）	★★★★
知识库优化	双向优化：query→doc生成扩展问题 / doc→query预生成可能提问	预处理阶段生成问答对 可提升后续检索准确率20%+	★★★★
知识图谱应用	Graph RAG通过实体识别+关系抽取构建语义网络	相比传统RAG：回答质量↑但 成本高 （预处理需30分钟/\$100）	★★★
联网搜索优化	8类需联网场景：时间/价格/营业状态/活动/天气/交通/预定/新闻	需生成 搜索关键词+平台建议+时间范围 三要素	★★★★
知识库健康检查	三维度检测：缺失知识/过期知识/冲突知识	通过测试问题集验证， 版本对比 准确率变化（如60%→100%）	★★★
Small-to-Big策略	先匹配摘要(small)再关联全文(big)	类似Google Notebook LM的实现方式， 降低长文档处理成本	★★★
运维智能体开发	框架选择：LangGraph/千帆Agent等	关键在 工具箱设计 而非框架本身	★★★
AI编程规范	逻辑设计 > 语法记忆，需关注：代码检查+需求拆解	禁止手敲代码 ，强调用AI生成（如DeepSeek）	★★★★